A Comprehensive Survey of Dataset Distillation

Shiye Lei and Dacheng Tao, *Fellow, IEEE*

(Survey paper)

***Abstract*—Deep learning technology has developed unprecedentedly in the last decade and has become the primary choice in many application domains. This progress is mainly attributed to a systematic collaboration in which rapidly growing computing resources encourage advanced algorithms to deal with massive data. However, it has gradually become challenging to handle the unlimited growth of data with limited computing power. To this end, diverse approaches are proposed to improve data processing efficiency. Dataset distillation, a dataset reduction method, addresses this problem by synthesizing a small typical dataset from substantial data and has attracted much attention from the deep learning community. Existing dataset distillation methods can be taxonomized into meta-learning and data matching frameworks according to whether they explicitly mimic the performance of target data. Although dataset distillation has shown surprising performance in compressing datasets, there are still several limitations such as distilling high-resolution data or data with complex label spaces. This paper provides a holistic understanding of dataset distillation from multiple aspects, including distillation frameworks and algorithms, factorized dataset distillation, performance comparison, and applications. Finally, we discuss challenges and promising directions to further promote future studies on dataset distillation.**

***Index Terms*—Efficient deep learning, neural network, data compression, dataset distillation.**

I. Introduction

DURING the past few decades, deep learning has achieved remarkable success due to powerful computing resources [1], [2], [3], [4], [5], which allow deep neural networks to directly handle giant datasets and bypass complicated manual feature extraction [6], [7]. For example, the powerful large language model GPT-3 contains 175 billion parameters and is trained on 45 terabytes of text data with thousands of graphics processing units (GPUs) [8]. However, massive data are generated every day [9], which pose a significant threat to training efficiency and data storage, and deep learning gradually reaches a bottleneck due to the mismatch between the volume of data and computing resources [10]. The emergence of *dataset distillation* (DD) is precisely to address this problem of large data volume. Dataset distillation, which was first proposed by [11] as a data compression technique that synthesizes a few high-informative data points to summarize numerous real data. Then, downstream models trained on the small synthetic dataset can achieve comparable generalization performance to those trained on the real data. A general illustration for dataset distillation is presented in Fig. 1.

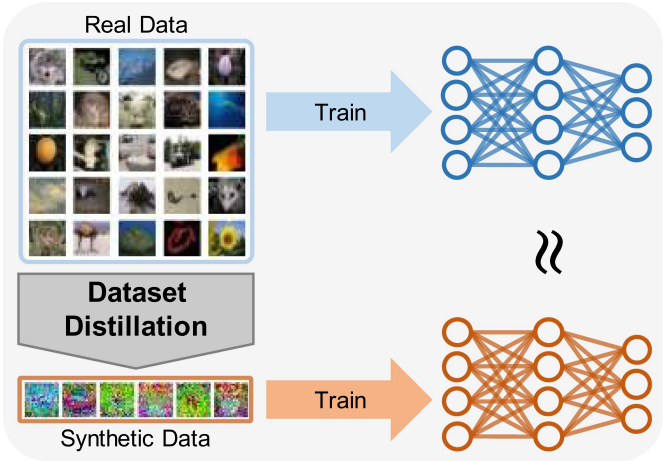


Fig. 1. Illustration of dataset distillation. The models trained on the large original dataset and small synthetic dataset demonstrate comparable performance on the test set.

Before the appearance of DD, coreset selection [12], [13], [14] plays a pivotal role in dataset reduction by selecting a few prototype examples from the original training dataset as the coreset, and then the model is solely trained on the small coreset to save training costs while avoiding large performance drops. However, elements in the coreset are unmodified and constrained by the original data, which considerably restricts the coreset's expressiveness, especially when the coreset budget is limited. Different from coreset selection, dataset distillation removes the restriction of uneditable elements and carefully modifies a small number of examples to preserve more information, as shown in Fig. 3 for synthetic examples. By distilling the knowledge of the large original dataset into a small synthetic set, models trained on the distilled dataset can acquire a better generalization performance compared to the uneditable coreset.

Due to the property of extremely high dimensions in the deep learning regime, the data information is hardly disentangled to specific concepts, and thus distilling numerous high-dimensional data into a few points is not a trivial task. Based on the objectives applied to mimic target data, dataset distillation methods can be grouped into meta-learning framework

Manuscript received 7 February 2023; revised 26 September 2023; accepted 4 October 2023. Date of publication 6 October 2023; date of current version 5 December 2023. This work was supported by Australian Research Council Project FL-170100117. Recommended for acceptance by M. Sugiyama. *(Corresponding author: Dacheng Tao.)*

The authors are with the Sydney AI Centre, School of Computer Science, Faculty of Engineering, The University of Sydney, Darlington, NSW 2008, Australia (e-mail: slei5230@uni.sydney.edu.au; dacheng.tao@sydney.edu.au).

Digital Object Identifier 10.1109/TPAMI.2023.3322540

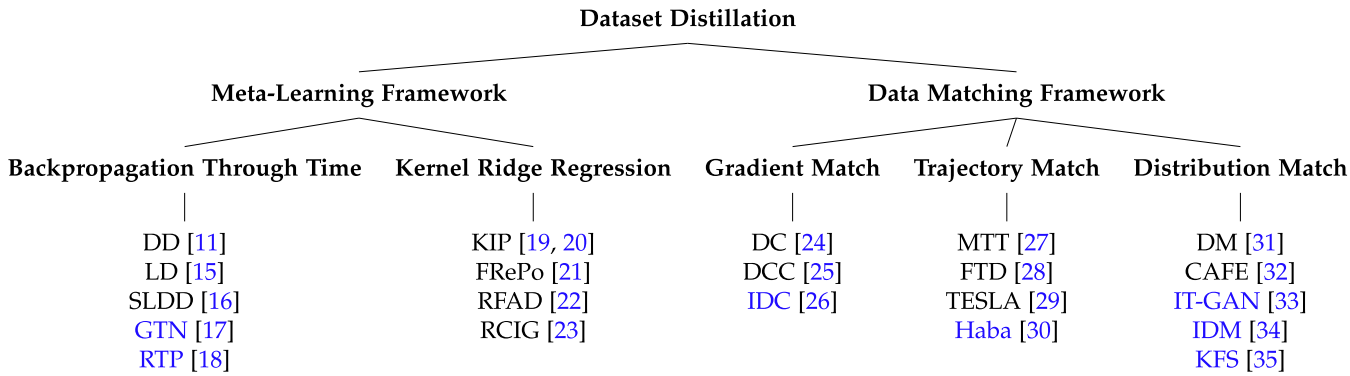


Fig. 2. Tree diagram for different categories of dataset distillation algorithms. The factorized DD methods are marked in blue.

(Section III) and data matching framework (Section IV), and these techniques in each framework can be further classified in a much more detailed manner. In the meta-learning framework, the distilled data are considered as hyperparameters and optimized in a nested loop fashion according to the distilled-data-trained model's risk with respect to (*w.r.t.*) the target data [11], [36]. The data matching framework updates distilled data by imitating the influence of target data on model training from parameter or feature space [24], [27], [31]. Fig. 2 presents these different categories of DD algorithms in a tree diagram.

Apart from directly considering the synthetic examples as optimization objectives, in some studies, a proxy model is designed, which consists of latent codes and decoders to generate highly informative examples and to resort to learning the latent codes and decoders (Section V). For example, Such et al. [17] employed a network to generate highly informative data from noise and optimized the network with the meta-learning framework. Zhao and Bilen [33] optimized the vectors and put them into the generator of a well-trained generative adversarial network (GAN) to produce the synthetic examples. Moreover, Deng and Russakovsky [18], Kim et al. [26], Liu et al. [30], Lee et al. [35] learned a couple of latent codes and decoders, and then synthetic data were generated according to the different combinations of latent codes and decodes. With this factorization of synthetic data, the compression ratio of DD can be further decreased, and the performance can also be improved due to the intraclass information extracted by latent codes.

In this paper, we present a comprehensive survey on dataset distillation, and the main objectives of this survey are as follows: 1) present a clear and systematic overview of dataset distillation; 2) review the recent advances in DD algorithms and discuss various applications in dataset distillation; 3) give a comprehensive performance comparison *w.r.t.* different dataset distillation algorithms; and 4) provide a detailed discussion on the limitation and promising directions of dataset distillation to benefit future studies. We note that there is a concurrent survey on dataset distillation [37]. The most notable difference between [37] and our survey is the taxonomy of DD. It plainly classifies DD algorithms into four different categories of meta-model matching, gradient matching, trajectory matching, and distribution matching, while our hierarchical taxonomy first introduces meta-learning and data matching frameworks, which are then followed by a more fine-grained division. Therefore, our taxonomy provides a more systematic framework to track and summarize the development of DD algorithms and better sheds light on future DD studies.

The rest of this paper is organized as follows. We first provide some background *w.r.t.* DD in Section II. DD methods under the meta-learning framework are described and comprehensively analyzed in Section III, and a description of the data matching framework follows in Section IV. Section V discusses different types of factorized dataset distillation. Next, we report the performance of various distillation algorithms in Section VI. Applications with dataset distillation are shown in Section VIII, and finally we discuss challenges and future directions in Section IX.

II. Background

Before introducing dataset distillation, we first define some notations used in this paper. For a dataset $\mathcal{T} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^m$, $\boldsymbol{x}_i \in \mathbb{R}^d$, d is the dimension of the input data, and y_i is the label. We assume that $(\boldsymbol{x}_i, y_i)$ for $1 \leq i \leq m$ are independent and identically distributed (i.i.d.) random variables drawn from the data generating distribution $\mathcal{D}$. We employ $f_{\boldsymbol{\theta}}$ to denote the neural network parameterized by $\boldsymbol{\theta}$, and $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ is the prediction or output of $f_{\boldsymbol{\theta}}$ at $\boldsymbol{x}$. Moreover, we also define the loss between the prediction and ground truth as $\ell(f_{\boldsymbol{\theta}}(\boldsymbol{x}), y)$, and the expected risk in terms of $\boldsymbol{\theta}$ is defined as

$$\mathcal{R}_{\mathcal{D}}(\boldsymbol{\theta}) = \mathbb{E}_{(\boldsymbol{x},y)\sim\mathcal{D}}\left[\ell\left(f_{\boldsymbol{\theta}}\left(\boldsymbol{x}\right), y\right)\right]. \tag{1}$$

Since the data generating distribution $\mathcal{D}$ is unknown, evaluating the expected risk $\mathcal{R}_{\mathcal{D}}$ is impractical. Therefore, a practical way to estimate the expected risk is by the empirical risk $\mathcal{R}_{\mathcal{T}}$, which is defined as

$$\mathcal{R}_{\mathcal{T}}(\boldsymbol{\theta}) = \mathbb{E}_{(\boldsymbol{x},y)\sim\mathcal{T}}\left[\ell\left(f_{\boldsymbol{\theta}}\left(\boldsymbol{x}\right), y\right)\right] = \frac{1}{m}\sum_{i=1}^{m}\ell\left(f_{\boldsymbol{\theta}}\left(\boldsymbol{x}_i\right), y_i\right). \tag{2}$$

For the training algorithm $\mathtt{alg}$, $\mathtt{alg}(\mathcal{T}, \boldsymbol{\theta}^{(0)})$ denotes the learned parameters returned by empirical risk minimization (ERM) on the dataset $\mathcal{T}$ with the initialized parameter $\boldsymbol{\theta}^{(0)}$

$$\mathtt{alg}\left(\mathcal{T}, \boldsymbol{\theta}^{(0)}\right) = \arg\min_{\boldsymbol{\theta}} \mathcal{R}_{\mathcal{T}}\left(\boldsymbol{\theta}\right). \tag{3}$$

In the deep learning paradigm, gradient descent is the dominant training algorithm to train a neural network by minimizing the empirical risk step by step. Specifically, the network's parameters are initialized with $\boldsymbol{\theta}^{(0)}$, and then the parameter is iteratively updated according to the gradient of empirical risk

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta \boldsymbol{g}_{\mathcal{T}}^{(k)}, \tag{4}$$

where η is the learning rate and $\boldsymbol{g}_{\mathcal{T}}^{(k)} = \nabla_{\boldsymbol{\theta}^{(k)}} \mathcal{R}_{\mathcal{T}}(\boldsymbol{\theta}^{(k)})$ is the gradient. We omit $\boldsymbol{\theta}^{(0)}$ and use $\mathtt{alg}(\mathcal{T})$ if there is no ambiguity for the sake of simplicity. Then, the model f trained on the dataset $\mathcal{T}$ can be denoted as $f_{\mathtt{alg}(\mathcal{T})}$.

Because deep learning models are commonly extremely overparameterized, i.e., the number of model parameters overwhelms the number of training examples, the empirical risk readily reaches zero. In this case, the generalization error, which measures the difference between the expected risk and the empirical risk, can be solely equal to the expected risk, which is reflected by test loss or test error in practical pipelines.

A. Formalizing Dataset Distillation

Given a target dataset (source training dataset) $\mathcal{T} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{m}$, the objective of dataset distillation is to extract the knowledge of $\mathcal{T}$ into a small synthetic dataset $\mathcal{S} = \{(\boldsymbol{s}_j, y_j)\}_{j=1}^{n}$, where $n \ll m$, and the model trained on the small distilled dataset $\mathcal{S}$ can achieve a comparable generalization performance to the large original dataset $\mathcal{T}$

$$\mathbb{E}_{\substack{(\boldsymbol{x},y)\sim\mathcal{D} \\ \boldsymbol{\theta}^{(0)}\sim\mathbf{P}}} \left[\ell\left(f_{\mathtt{alg}(\mathcal{T})}(\boldsymbol{x}), y\right)\right] \simeq \mathbb{E}_{\substack{(\boldsymbol{x},y)\sim\mathcal{D} \\ \boldsymbol{\theta}^{(0)}\sim\mathbf{P}}} \left[\ell\left(f_{\mathtt{alg}(\mathcal{S})}(\boldsymbol{x}), y\right)\right]. \tag{5}$$

Because the training algorithm $\mathtt{alg}(\mathcal{S}, \boldsymbol{\theta}^{(0)})$ is determined by the training set $\mathcal{S}$ and the initialized network parameter $\boldsymbol{\theta}^{(0)}$, many dataset distillation algorithms will take expectation on $\mathcal{S}$ *w.r.t.* $\boldsymbol{\theta}^{(0)}$ to improve the robustness of the distilled dataset $\mathcal{S}$ to different parameter initialization, and the objective function has the form of $\mathbb{E}_{\boldsymbol{\theta}^{(0)}\sim\mathbf{P}}[\mathcal{L}(\mathcal{S})]$. In the following narrative, we omit this expectation *w.r.t.* initialization $\boldsymbol{\theta}^{(0)}$ for the sake of simplicity.

III. Meta-Learning Framework

By assuming $\mathcal{R}_{\mathcal{D}}(\mathtt{alg}(\mathcal{S})) \simeq \mathcal{R}_{\mathcal{T}}(\mathtt{alg}(\mathcal{S}))$, the target dataset $\mathcal{T}$ is employed as the validation set in terms of the model $\mathtt{alg}(\mathcal{S})$, and in consequence, the objective of DD can be converted to minimize $\mathcal{R}_{\mathcal{T}}(\mathtt{alg}(\mathcal{S}))$ to improve the generalization performance of $\mathtt{alg}(\mathcal{S})$ in terms of $\mathcal{D}$. To this end, dataset distillation falls into the meta-learning area: the hyperparameter $\mathcal{S}$ is updated by the meta (or outer) algorithm, and the base (or inner) algorithm solves the conventional supervised learning problem *w.r.t.* the synthetic dataset $\mathcal{S}$ [38]; and the dataset distillation task can be formulated to a bilevel optimization problem as follows:

$$\mathcal{S}^* = \arg\min_{\mathcal{S}} \mathcal{R}_{\mathcal{T}}(\mathtt{alg}(\mathcal{S})) \quad \text{(outer loop)}, \tag{6}$$

subject to

$$\mathtt{alg}(\mathcal{S}) = \arg\min_{\boldsymbol{\theta}} \mathcal{R}_{\mathcal{S}}(\boldsymbol{\theta}) \quad \text{(inner loop)}. \tag{7}$$

The inner loop optimizes the model parameters based on the synthetic dataset and is often realized by gradient descent for neural networks or regression for kernel method. During the outer loop iteration, the synthetic set is updated by minimizing the model's risk in terms of the target dataset. With the nested loop, the synthetic dataset gradually converges to one of the optima.

According to the model and optimization methods used in the inner loop, the meta-learning framework of DD can be further classified into two sub-categories of backpropagation through time time (BPTT) approach and kernel ridge regression (KRR) approach.

A. Backpropagation Through Time Approach

As shown in the above formulation of bilevel optimization, the objective function of DD can be directly defined as the meta-loss of

$$\mathcal{L}(\mathcal{S}) = \mathcal{R}_{\mathcal{T}}(\mathtt{alg}(\mathcal{S})). \tag{8}$$

Then the distilled dataset is updated by $\mathcal{S} = \mathcal{S} - \alpha \nabla_{\mathcal{S}} \mathcal{L}(\mathcal{S})$ with the step size α. However, in the inner loop of (7), the neural network is trained by iterative gradient descent ((4)), which yields a series of intermediate parameter states of $\{\boldsymbol{\theta}^{(0)}, \boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(T)}\}$, backpropagation through time [39] is required to recursively compute the meta-gradient $\nabla_{\mathcal{S}} \mathcal{L}(\mathcal{S})$

$$\nabla_{\mathcal{S}} \mathcal{L}(\mathcal{S}) = \frac{\partial \mathcal{L}}{\partial \mathcal{S}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}^{(T)}} \left(\sum_{k=0}^{k=T} \frac{\partial \boldsymbol{\theta}^{(T)}}{\partial \boldsymbol{\theta}^{(k)}} \cdot \frac{\partial \boldsymbol{\theta}^{(k)}}{\partial \mathcal{S}} \right), \tag{9}$$

and

$$\frac{\partial \boldsymbol{\theta}^{(T)}}{\partial \boldsymbol{\theta}^{(k)}} = \prod_{i=k+1}^{T} \frac{\partial \boldsymbol{\theta}^{(i)}}{\partial \boldsymbol{\theta}^{(i-1)}}. \tag{10}$$

We present the meta-gradient backpropagation of BPTT in Fig. 4(a). Due to the requirement of unrolling the recursive computation graph, BPTT is both computationally expensive and memory demanding, which also severely affects the final distillation performance.

To alleviate the inefficiency in unrolling the long parameter path of $\{\boldsymbol{\theta}^{(0)}, \ldots, \boldsymbol{\theta}^{(T)}\}$, Wang et al. [11] adopted a single-step optimization *w.r.t.* the model parameter from $\boldsymbol{\theta}^{(0)}$ to $\boldsymbol{\theta}^{(1)}$, and the meta-loss was computed based on $\boldsymbol{\theta}^{(1)}$ and the target dataset $\mathcal{T}$

$$\boldsymbol{\theta}^{(1)} = \boldsymbol{\theta}^{(0)} - \eta \nabla_{\boldsymbol{\theta}^{(0)}} \mathcal{R}_{\mathcal{S}}(\boldsymbol{\theta}^{(0)}) \quad \text{and} \quad \mathcal{L} = \mathcal{R}_{\mathcal{T}}(\boldsymbol{\theta}^{(1)}). \tag{11}$$

Therefore, the distilled data $\mathcal{S}$ and learning rate η can be efficiently updated via the short-range BPTT as follows:

$$\mathcal{S} = \mathcal{S} - \alpha_{\boldsymbol{s}_i} \nabla \mathcal{L} \quad \text{and} \quad \eta = \eta - \alpha_{\eta} \nabla \mathcal{L}. \tag{12}$$

Unlike freezing distilled labels, Sucholutsky and Schonlau [16] extended the work of Wang et al. [11] by learning a soft-label in the synthetic dataset $\mathcal{S}$: the label y in the synthetic dataset is also trainable for better information compression, i.e., $y_i = y_i - \alpha \nabla_{y_i} \mathcal{L}$ for $(\boldsymbol{s}_i, y_i) \in \mathcal{S}$. Similarly, Bohdal et al. [15] also extended the standard example distillation to label distillation by solely optimizing the labels of synthetic datasets. Moreover, these researchers provided improvements on the efficiency of long inner loop optimization via 1) iteratively updating the model parameters $\boldsymbol{\theta}$ and the distilled labels y, i.e., one outer step

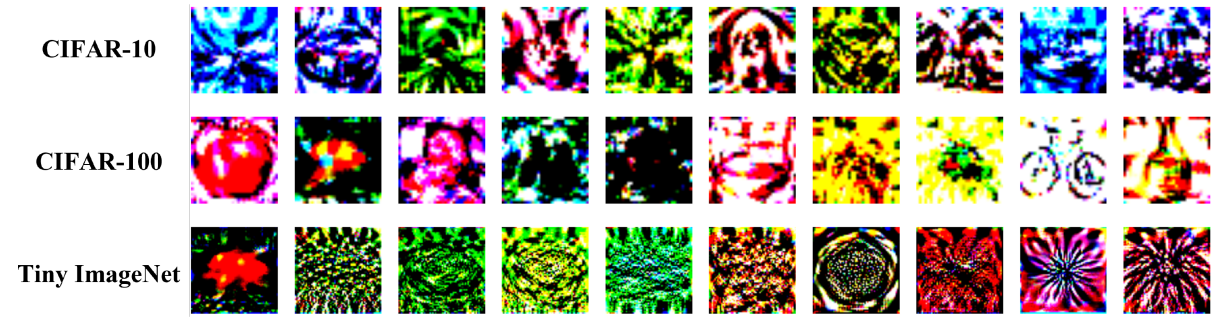


Fig. 3. Example synthetic images distilled from CIFAR-10/100 and Tiny ImageNet by matching the training trajectory [27].

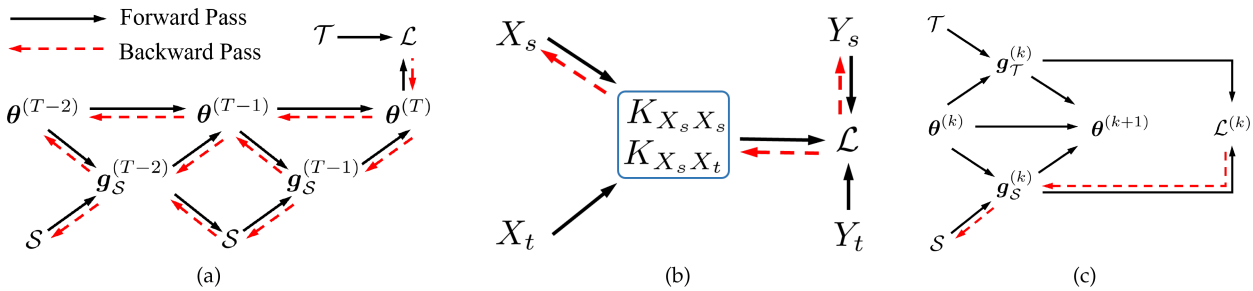


Fig. 4. (a) Meta-gradient backpropagation in BPTT [21]; (b) Meta-gradient backpropagation in kernel ridge regression; and (c) Meta-gradient backpropagation in gradient matching. The gradient $g_{\mathcal{S}}^{(T)} = \nabla_{\boldsymbol{\theta}^{(T)}} \mathcal{R}_{\mathcal{S}}(\boldsymbol{\theta}^{(T)})$.

followed by only one inner step for faster convergence; and 2) fixing the feature extractor of neural networks and solely updating the last linear layer with ridge regression to avoid second-order meta-gradient computation. Although the BPTT framework has been shown to underperform other algorithms, Deng and Russakovsky [18] empirically demonstrated that adding momentum term and longer unrolled trajectory (200 steps) in the inner loop optimization can considerably enhance the distillation performance, and the inner loop of model training becomes

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta \boldsymbol{m}^{(k+1)}, \tag{13}$$

where

$$\boldsymbol{m}^{(k+1)} = \beta \boldsymbol{m}^{(k)} + \nabla_{\boldsymbol{\theta}^{(k)}} \mathcal{R}_{\mathcal{T}}(\boldsymbol{\theta}^{(k)}) \quad \text{s.t.} \quad \boldsymbol{m}^{(0)} = \mathbf{0}. \tag{14}$$

B. Kernel Ridge Regression Approach

Although multistep gradient descent can gradually approach the optimal network parameters in terms of the synthetic dataset during the inner loop, this iterative algorithm makes the meta-gradient backpropagation highly inefficient, as shown in BPTT. Considering the existence of closed-form solutions in the kernel regression regime, Nguyen et al. [19] replaced the neural network in the inner loop with a kernel model, which bypasses the recursive backpropagation of the meta-gradient. For the regression model $f(\boldsymbol{x}) = \boldsymbol{w}^\top \psi(\boldsymbol{x})$, where $\psi(\cdot)$ is a nonlinear mapping and the corresponding kernel is $K(\boldsymbol{x}, \boldsymbol{x}') = \langle \psi(\boldsymbol{x}), \psi(\boldsymbol{x}') \rangle$, there exists a closed-form solution for $\boldsymbol{w}$ when the regression model is trained on $\mathcal{S}$ with kernel ridge regression (KRR)

$$\boldsymbol{w} = \psi(X_s)^\top \left(\mathbf{K}_{X_s X_s} + \lambda I\right)^{-1} y_s, \tag{15}$$

where $\mathbf{K}_{X_s X_s} = [K(\boldsymbol{s}_i, \boldsymbol{s}_j)]_{ij} \in \mathbb{R}^{n \times n}$ is called the *kernel matrix* or *Gram matrix* associated with K and the dataset $\mathcal{S}$, and $\lambda > 0$ is a fixed regularization parameter [40]. Therefore, the mean square error (MSE) of predicting $\mathcal{T}$ with the model trained on $\mathcal{S}$ is

$$\mathcal{L}(\mathcal{S}) = \frac{1}{2} \left\| y_t - \mathbf{K}_{X_t X_s} \left(\mathbf{K}_{X_s X_s} + \lambda I\right)^{-1} y_s \right\|^2, \tag{16}$$

where $\mathbf{K}_{X_t X_s} = [K(\boldsymbol{x}_i, \boldsymbol{s}_j)]_{ij} \in \mathbb{R}^{m \times n}$. Then the distilled dataset is updated via the meta-gradient of the above loss. Due to the closed-form solution in KRR, $\boldsymbol{\theta}$ does not require an iterative update and the backward pass of the gradient thus bypasses the recursive computation graph, as shown in Fig. 4(b).

In the KRR regime, the synthetic dataset $\mathcal{S}$ can be directly updated by backpropagating the meta-gradient through the kernel function. Although this formulation is solid, this algorithm is designed in the KRR scenario and only employs simple kernels, which causes performance drops when the distilled dataset is transferred to train neural networks. Jacot et al. [41] proposed the neural tangent kernel (NTK) theory that proves the equivalence between training infinite-width neural networks and kernel regression. With this equivalence, Nguyen et al. [20] employed infinite-width networks as the kernel for dataset distillation, which narrows the gap between the scenarios of KRR and deep learning.

However, every entry in the kernel matrix must be calculated separately via the kernel function, and thus computing the kernel matrix $\mathbf{K}_{X_sX_t}$ has the time complexity of $\mathcal{O}(|\mathcal{T}||\mathcal{S}|)$, which is severely inefficient for large-scale datasets with huge $|\mathcal{T}|$. To tackle this problem, Loo et al. [22] replaced the NTK kernel with neural network Gaussian process (NNGP) kernel that only considers the training dynamic of the last-layer classifier for speed up. With this replacement, the random features $\psi(\boldsymbol{x})$ and $\psi(\boldsymbol{s})$ can be explicitly computed via multiple sampling from the Gaussian process, and thus the kernel matrix computation can be decomposed into random feature calculation and random feature matrix multiplication. Because matrix multiplication requires negligible amounts of time for small distilled datasets, the time complexity of kernel matrix computation degrades to $\mathcal{O}(|\mathcal{T}|+|\mathcal{S}|)$. In addition, these authors demonstrated two issues of MSE loss ((16)) used in [19] and [20]: (1) over-influence on corrected data: the correctly classified examples in $\mathcal{T}$ can induce larger loss than the incorrectly classified examples; and (2) unclear probabilistic interpretation for classification tasks. To overcome these issues, they propose to apply a cross-entropy (CE) loss with Platt scaling [42] to replace the MSE loss

$$\mathcal{L}_\tau = \mathrm{CE}(y_t, \hat{y}_t/\tau), \tag{17}$$

where τ is a positive learned temperature scaling parameter, and the prediction $\hat{y}_t = \mathbf{K}_{X_tX_s}(\mathbf{K}_{X_sX_s}+\lambda I)^{-1}y_s$ is still calculated using the KRR formula.

A similar efficient method was also proposed by Zhou et al. [21], which also focused on solving the last-layer classifier in neural networks with KRR. Specifically, the neural network $f_{\boldsymbol{\theta}} = g_{\boldsymbol{\theta}_2} \circ h_{\boldsymbol{\theta}_1}$ can be decomposed with the feature extractor h and the linear classifier g. Then these coworkers fixed the feature extractor h and the linear classifier g possesses a closed-form solution with KRR, and the distilled data are accordingly optimized with the MSE loss

$$\mathcal{L}(\mathcal{S}) = \frac{1}{2}\left\| y_t - \mathbf{K}^{\boldsymbol{\theta}_1^{(k)}}_{X_tX_s}\left(\mathbf{K}^{\boldsymbol{\theta}_1^{(k)}}_{X_sX_s}+\lambda I\right)^{-1} y_s \right\|^2, \tag{18}$$

and

$$\boldsymbol{\theta}_1^{(k)} = \boldsymbol{\theta}_1^{(k-1)} - \eta\nabla_{\boldsymbol{\theta}_1^{(k-1)}}\mathcal{R}_{\mathcal{S}}(\boldsymbol{\theta}_1^{(k-1)}), \tag{19}$$

where the kernel matrices $\mathbf{K}^{\boldsymbol{\theta}_1}_{X_tX_s}$ and $\mathbf{K}^{\boldsymbol{\theta}_1}_{X_sX_s}$ are induced by $h_{\boldsymbol{\theta}_1}(\cdot)$. Notably, although the feature extractor $h_{\boldsymbol{\theta}_1}$ is continuously updated, the classifier $g_{\boldsymbol{\theta}_2}$ is directly solved by KRR, and thus the meta-gradient backpropagation side-steps the recursive computation graph.

C. Discussion

From the loss surface perspective [43], the meta-learning framework of minimizing $\mathcal{R}_{\mathcal{T}}(\mathtt{alg}(\mathcal{S}))$ can be considered to mimic the local minima of target data with the distilled data. However, the loss landscape *w.r.t.* parameters is closely related to the network architecture, while only one type of small network is used in the BPTT approach. Consequently, there is a moderate performance drop when the distilled dataset is employed to train other complicated networks. Moreover, a long unrolled trajectory and second-order gradient computation are also two key challenges for BPTT approach, which hinder its efficiency. The KRR approach compensates for these shortcomings by replacing networks with the nonparametric kernel model which admits closed-form solution. Although KRR is nonparametric and does not involve neural networks during the distillation process, previous research has shown that the training dynamic of neural networks is equal to the kernel method when the width of networks becomes infinite [41], [44], [45], [45], which partially guarantees the feasibility of the kernel regression approach and explains its decent performance when transferred to wide neural networks.

IV. Data Matching Framework

Although it is not feasible to explicitly extract information from target data and then inject them into synthetic data, information distillation can be achieved by implicitly aligning the byproducts of target data and synthetic data from different aspects; and this byproduct matching allows synthetic data to imitate the influence of target data on model training. The objective function of data matching can be summarized as follows.

$$\mathcal{L}(\mathcal{S}) = \sum_{k=0}^{T} D\left(\phi\left(\mathcal{S}, \boldsymbol{\theta}^{(k)}\right), \phi\left(\mathcal{T}, \boldsymbol{\theta}^{(k)}\right)\right), \tag{20}$$

subject to

$$\boldsymbol{\theta}^{(k)} = \boldsymbol{\theta}^{(k-1)} - \eta\nabla_{\boldsymbol{\theta}^{(k-1)}}\mathcal{R}_{\mathcal{S}}\left(\boldsymbol{\theta}^{(k-1)}\right), \tag{21}$$

where $D(\cdot,\cdot)$ is a distance function, and $\phi(\cdot)$ maps the dataset $\mathcal{S}$ or $\mathcal{T}$ to other informative spaces, such as gradient, parameter, and feature spaces. In practical implementation, the full datasets of $\mathcal{S}$ and $\mathcal{T}$ are often replaced with randomly sampled batches of $\mathcal{B}_{\mathcal{S}}$ and $\mathcal{B}_{\mathcal{T}}$ for memory saving and faster convergence.

Compared to the aforementioned meta-learning framework, the data matching loss not only focuses on the final parameter $\mathtt{alg}(\mathcal{S})$ but also supervises the intermediate states, as shown in the sum operation $\sum_{k=0}^{T}$. By this, the distilled data can better imitate the influence of target data on training networks at different training stages.

A. Gradient Matching Approach

To achieve comparable generalization performance, a natural approach is to imitate the model parameters, i.e., matching the training trajectories introduced by $\mathcal{S}$ and $\mathcal{T}$. With a fixed parameter initialization, the training trajectory of $\{\boldsymbol{\theta}^{(0)}, \boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(T)}\}$ is equal to a series of gradients $\{\boldsymbol{g}^{(0)}, \ldots, \boldsymbol{g}^{(T)}\}$. Therefore, matching the gradients induced by $\mathcal{S}$ and $\mathcal{T}$ is a convincing proxy to mimic the influence on model parameters [24], and the

objective function is formulated as

$$\mathcal{L}(\mathcal{S}) = \sum_{k=0}^{T-1} D\left(\nabla_{\boldsymbol{\theta}^{(k)}}\mathcal{R}_{\mathcal{S}}\left(\boldsymbol{\theta}^{(k)}\right), \nabla_{\boldsymbol{\theta}^{(k)}}\mathcal{R}_{\mathcal{T}}\left(\boldsymbol{\theta}^{(k)}\right)\right). \quad (22)$$

The difference D between gradients is measured in the layer-wise aspect

$$D\left(\boldsymbol{g}_{\mathcal{S}}, \boldsymbol{g}_{\mathcal{T}}\right) = \sum_{l=1}^{L} \mathtt{dis}\left(\boldsymbol{g}_{\mathcal{S}}^{l}, \boldsymbol{g}_{\mathcal{T}}^{l}\right), \quad (23)$$

where $\boldsymbol{g}^l$ denotes the gradient of ith layer, and $\mathtt{dis}$ is the sum of cosine distance as follows:

$$\mathtt{dis}\left(\boldsymbol{A}, \boldsymbol{B}\right) = \sum_{i=1}^{\mathtt{out}}\left(1 - \frac{\boldsymbol{A}_i\boldsymbol{B}_i}{\|\boldsymbol{A}_i\|\|\boldsymbol{B}_i\|}\right), \quad (24)$$

where $\mathtt{out}$ denotes the number of output channels for specific layer; and $\boldsymbol{A}_i$ and $\boldsymbol{B}_i$ are the flatten gradients in the ith channel.

To improve the convergence speed in practical implementation, Zhao et al. [24] proposed to match gradient in class-wise as follows:

$$\mathcal{L}^{(k)} = \sum_{c=1}^{C} D\left(\nabla_{\boldsymbol{\theta}^{(k)}}\mathcal{R}\left(\mathcal{B}_{\mathcal{S}}^{c}, \boldsymbol{\theta}^{(k)}\right), \nabla_{\boldsymbol{\theta}^{(k)}}\mathcal{R}\left(\mathcal{B}_{\mathcal{T}}^{c}, \boldsymbol{\theta}^{(k)}\right)\right), \quad (25)$$

where c is the class index and $\mathcal{B}_{\mathcal{S}}^{c}$ and $\mathcal{B}_{\mathcal{T}}^{c}$ denotes the batch examples belong to the cth class. However, according to [25], the class-wise gradient matching pays much attention to the class-common features and overlooks the class-discriminative features in the target dataset, and the distilled synthetic dataset $\mathcal{S}$ does not possess enough class-discriminative information, especially when the target dataset is fine-grained, i.e., class-common features are dominant. Based on this finding, these coworkers proposed an improved objective function of

$$\mathcal{L}^{(k)} = D\left(\sum_{c=1}^{C}\nabla_{\boldsymbol{\theta}^{(k)}}\mathcal{R}\left(\mathcal{B}_{\mathcal{S}}^{c}, \boldsymbol{\theta}^{(k)}\right), \sum_{c=1}^{C}\nabla_{\boldsymbol{\theta}^{(k)}}\mathcal{R}\left(\mathcal{B}_{\mathcal{T}}^{c}, \boldsymbol{\theta}^{(k)}\right)\right), \quad (26)$$

to better capture contrastive signals between different classes through the sum of loss gradients between classes. A similar approach was proposed by Jiang et al. [47], which employed both intraclass and interclass gradient matching by combining (25) and (26). Moreover, these researchers measure the difference of gradients by considering the magnitude instead of only considering the angle, i.e., the cosine distance, and the distance function of (24) is improved to

$$\mathtt{dis}\left(\boldsymbol{A}, \boldsymbol{B}\right) = \sum_{i=1}^{\mathtt{out}}\left(1 - \frac{\boldsymbol{A}_i\boldsymbol{B}_i}{\|\boldsymbol{A}_i\|\|\boldsymbol{B}_i\|} + \|\boldsymbol{A}_i - \boldsymbol{B}_i\|\right). \quad (27)$$

To alleviate easy overfitting on the small dataset $\mathcal{S}$, Kim et al. [26] proposed to perform inner loop optimization on the target dataset $\mathcal{T}$ instead of $\mathcal{S}$, i.e., replaced the parameter update of (21) with

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta\nabla_{\boldsymbol{\theta}^{(k)}}\mathcal{R}_{\mathcal{T}}\left(\boldsymbol{\theta}^{(k)}\right). \quad (28)$$

Although data augmentation facilitates a large performance increase in conventional network training, conducting augmentation on distilled datasets demonstrates no improvement on the final test accuracy, because the characteristics of synthetic images are different from those of natural images and are not optimized under the supervision of various transformations. To leverage data augmentation on synthetic datasets, Zhao and Bilen [48] designed data Siamese augmentation (DSA) that homologously augments the distilled data and the target data during the distillation process as follows:

$$\mathcal{L} = D\left(\nabla_{\boldsymbol{\theta}}\mathcal{R}\left(\mathcal{A}_{\omega}\left(\mathcal{B}_{\mathcal{S}}\right), \boldsymbol{\theta}\right), \nabla_{\boldsymbol{\theta}}\mathcal{R}\left(\mathcal{A}_{\omega}\left(\mathcal{B}_{\mathcal{T}}\right), \boldsymbol{\theta}\right)\right), \quad (29)$$

where $\mathcal{A}$ is a family of image transformations such as cropping and flipping that are parameterized with $\omega^{\mathcal{S}}$ and $\omega^{\mathcal{T}}$ for synthetic and target data, respectively. In DSA, the augmented form of distilled data has a consistent correspondence *w.r.t.* the augmented form of the target data, i.e., $\omega^{\mathcal{S}} = \omega^{\mathcal{T}} = \omega$; and ω is randomly picked from $\mathcal{A}$ at different iterations. Notably, the transformation $\mathcal{A}$ requires to be differentiable *w.r.t.* the synthetic dataset $\mathcal{S}$ for backpropagation

$$\frac{\partial D(\cdot)}{\partial \mathcal{S}} = \frac{\partial D(\cdot)}{\partial \nabla_{\theta}\mathcal{L}(\cdot)}\frac{\partial \nabla_{\theta}\mathcal{L}(\cdot)}{\partial \mathcal{A}(\cdot)}\frac{\partial \mathcal{A}(\cdot)}{\partial \mathcal{S}}. \quad (30)$$

Through setting $\omega^{\mathcal{S}} = \omega^{\mathcal{T}}$, DSA permits the knowledge transfer from the transformation of target images to the corresponding transformation of synthetic images. Consequently, the augmented synthetic images also possess meaningful characteristics of the natural images. Due to its superior compatibility, DSA has been widely used in many data matching methods.

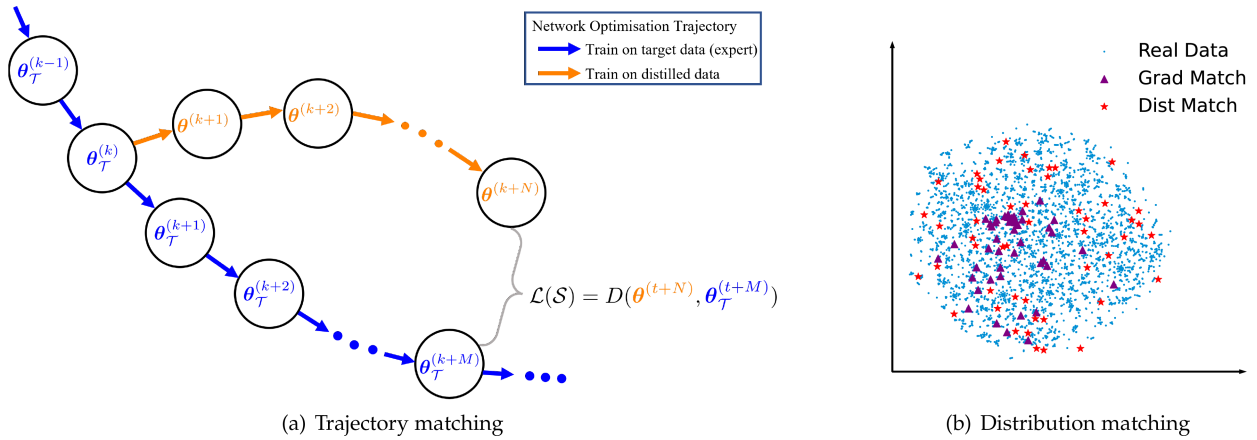
B. Trajectory Matching Approach

Unlike circuitously matching the gradients, Cazenavette et al. [27] directly matched the long-range training trajectory between the target dataset and the synthetic dataset. Specifically, they train models on the target dataset $\mathcal{T}$ and collect the expert training trajectory into a buffer in advance. Then the ingredients in the buffer are randomly selected to initialize the networks for training $\mathcal{S}$. After collecting the trajectories of $\mathcal{S}$, the synthetic dataset is updated by matching their parameters, as shown in Fig. 5(a). The objective loss of trajectory matching is defined as

$$\mathcal{L} = \frac{\|\boldsymbol{\theta}^{(k+N)} - \boldsymbol{\theta}_{\mathcal{T}}^{(k+M)}\|_2^2}{\|\boldsymbol{\theta}_{\mathcal{T}}^{(k)} - \boldsymbol{\theta}_{\mathcal{T}}^{(k+M)}\|_2^2}, \quad (31)$$

where $\boldsymbol{\theta}_{\mathcal{T}}$ denotes the target parameter by training the model on $\mathcal{T}$, which is stored in the buffer, and $\boldsymbol{\theta}^{(k+N)}$ are the parameter obtained by training the model on $\mathcal{S}$ for N epochs with the initialization of $\boldsymbol{\theta}_{\mathcal{T}}^{(k)}$. The denominator in the loss function is for normalization.

Although trajectory matching demonstrated empirical success, Du et al. [28] argued that $\boldsymbol{\theta}^{(k)}$ is generally induced by training on $\mathcal{S}$ for k epochs during the natural learning process, while it was directly initialized with $\boldsymbol{\theta}_{\mathcal{T}}^{(k)}$ in trajectory matching, which causes the *accumulated trajectory error* that measures the difference between the parameters from real trajectories of $\mathcal{S}$ and $\mathcal{T}$. Because the accumulated trajectory error is induced by the



(a) Trajectory matching (b) Distribution matching

Fig. 5. (a) An illustration of trajectory matching [27]; (b) T-SNE [46] visualization on the first class of CIFAR-10. Compared to gradient matching (Grad Match, marked with purple triangles), distribution matching (Dist Match, marked with red stars) can more comprehensively cover the real data distribution in feature space.

mismatch between $\boldsymbol{\theta}_{\mathcal{T}}^{(k)}$ and real $\boldsymbol{\theta}^{(k)}$ from the natural training, these researchers alleviate this error by adding random noise to when initializing $\boldsymbol{\theta}^{(k)}$ to improve robustness *w.r.t.* $\boldsymbol{\theta}_{\mathcal{T}}^{(k)}$.

Compared to matching gradients, while trajectory matching side-steps second-order gradient computation, unfortunately, unrolling N SGD updates are required during meta-gradient backpropagation due to the existence of $\boldsymbol{\theta}^{(k+N)}$ in (31). The unrolled gradient computation significantly increases the memory burden and impedes scalability. By disentangling the meta-gradient *w.r.t.* synthetic examples into two passes, Cui et al. [29] greatly reduced the memory required by trajectory matching and successfully scaled trajectory matching approach to the large ImageNet-1K dataset [49]. Motivated by knowledge distillation [50], these scholars also proposed assigning soft-labels to synthetic examples with pretrained models in the buffer; and the soft-labels help learn intraclass information and consequently improve distillation performance.

C. Distribution Matching Approach

Although the above parameter-wise matching shows satisfying performance, Zhao and Bilen [31] visualized the distilled data in two-dimensional plane and revealed that there is a large distribution discrepancy between the distilled data and the target data. In other words, the distilled dataset cannot comprehensively cover the data distribution in feature space, as shown in Fig. 5(b). Based on this discovery, these researchers proposed to match the synthetic and target data from the distribution perspective for dataset distillation. Specifically, they employed the pretrained feature extractor $\psi_{\boldsymbol{v}}$ with the parameter $\boldsymbol{v}$ to achieve mapping from the input space to the feature space. The synthetic data are optimized according to the objective function

$$\mathcal{L}(\mathcal{S}) = \sum_{c=1}^{C} \|\psi_{\boldsymbol{v}}(\mathcal{B}_{\mathcal{S}}^{c}) - \psi_{\boldsymbol{v}}(\mathcal{B}_{\mathcal{T}}^{c})\|^2, \tag{32}$$

where c denotes the class index. An illustration of distribution matching is also presented in Fig. 6. As shown in (32), distribution matching does not rely on the model parameters and drops

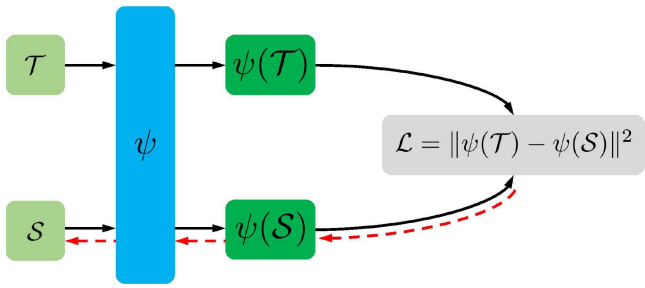


Fig. 6. Illustration of distribution matching. ψ is the feature extractor, and red dashed lines denote the backpropagation of gradients.

bilevel optimization for less memory requirement, whereas it empirically underperforms the above gradient and trajectory matching approaches.

Wang et al. [32] improved the distribution matching from several aspects: 1) using multiple-layer features other than only the last-layer features for matching, and the feature matching loss is formulated as follows:

$$\mathcal{L}_{\mathrm{f}} = \sum_{c=1}^{C} \sum_{l=1}^{L} |f_{\boldsymbol{\theta}}^{l}(\mathcal{B}_{\mathcal{S}}^{c}) - f_{\boldsymbol{\theta}}^{l}(\mathcal{T}_{\mathcal{S}}^{c})|^2, \tag{33}$$

where $f_{\boldsymbol{\theta}}^{l}$ denotes the lth layer features and L is the total number of network layers; 2) recalling the bilevel optimization that updates $\mathcal{S}$ with different model parameters by inserting the inner loop of $\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta \nabla_{\boldsymbol{\theta}^{(k)}} \mathcal{R}_{\mathcal{S}}(\boldsymbol{\theta}^{(k)})$; and 3) proposing the discrimination loss in the last-layer feature space to enlarge the class distinction of synthetic data. Specifically, the synthetic feature center $\bar{f}_{c,L}^{\mathcal{S}}$ of each category c is obtained by averaging the batch $\mathcal{B}_{\mathcal{S}}^{c}$. The objective of discrimination loss is to improve the classification ability of the feature center $\bar{\boldsymbol{F}}_{L}^{\mathcal{S}} = [\bar{f}_{1,L}^{\mathcal{S}}, \bar{f}_{2,L}^{\mathcal{S}}, \ldots, \bar{f}_{C,L}^{\mathcal{S}}]$ on predicting the real data feature $\boldsymbol{F}_{L}^{\mathcal{T}} = [f_{1,L}^{\mathcal{T}}, f_{1,L}^{\mathcal{T}}, \ldots, f_{C,L}^{\mathcal{T}}]$. The logits are first calculated by

$$\mathbf{O} = \left\langle \boldsymbol{F}_{L}^{\mathcal{T}}, \left(\bar{\boldsymbol{F}}_{L}^{\mathcal{S}}\right)^{\top} \right\rangle, \tag{34}$$

where $\mathbf{O} \in \mathbb{R}^{N\times C}$ contains the logits of $N = C \times |\mathcal{B}_{\mathcal{T}}^{c}|$ target data points. Then the probability p_i in terms of the ground-truth label is derived via $p_i = \text{softmax}(\mathbf{O}_i)$; and the classification loss is

$$\mathcal{L}_{\text{d}} = \frac{1}{N}\sum_{i=1}^{N} \log p_i. \tag{35}$$

Therefore, the total loss in [32] for dataset distillation is $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{f}} + \beta\mathcal{L}_{\text{d}}$, where β is a positive factor for balancing the feature matching and discrimination loss. Similar to the discrimination loss, Zhao et al. [34] add a classification loss $\mathcal{L}_{CE}$ as regularization to mitigate less classified synthetic data caused by the first-order moment mean matching

$$\mathcal{L}_{CE}(\mathcal{S}) = \sum_{i=1}^{|\mathcal{S}|} CE(f_{\boldsymbol{\theta}}(s_i), y_i), \tag{36}$$

where the network parameter $\boldsymbol{\theta}$ is varied in different loops.

D. Discussion

The gradient matching approach can be considered to be short-range parameter matching, while its backpropagation requires second-order gradient computation. Although the trajectory matching approach makes up for this imperfection, the long-range trajectory introduces a recursive computation graph during meta-gradient backpropagation. Different from matching in parameter space, distribution matching approach employs the feature space as the match proxy and also bypasses second-order gradient computation. Although distribution matching has advantages in scalability *w.r.t.* high-dimensional data, it empirically underperforms trajectory matching, which might be attributed to the mismatch between the comprehensive distribution and decent distillation performance. In detail, distribution matching achieves DD by mimicking features of the target data, so that the distilled data are evenly distributed in the feature space. However, not all features are equally important and distribution matching might waste the budget on imitating less informative features, thereby undermining the distillation performance.

V. Factorized Dataset Distillation

In representation learning, although the image data are in the extremely high-dimensional space, they may lie on a low-dimensional manifold and rely on a few features, and one can recover the source image from the low-dimensional features with specific decoders [51], [52]. For this reason, it is plausible to implicitly learn synthetic datasets by optimizing their factorized features and corresponding decoders, which is termed as *factorized dataset distillation*, as shown in Fig. 7. To be in harmony with decoders, we recall the notion of a feature as code in terms of the dataset. By factorizing synthetic datasets with a combination of codes and decoders, the compression ratio can be further decreased, and information redundancy in distilled images can also be reduced. According to the learnability of the code and decoder, we classify factorized dataset distillation into three categories of code-based DD, decoder-based DD, and code-decoder DD.

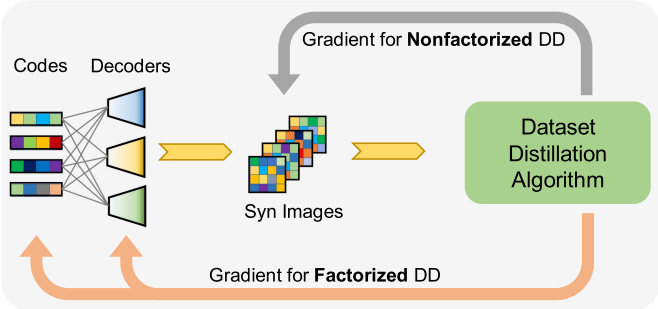


Fig. 7. Schematic diagrams of nonfactorized dataset distillation and factorized dataset distillation. Unlike directly updating synthetic (Syn) images in nonfactorized DD, factorized DD optimizes the codes and decoders, which are then combined to generate synthetic data.

Code-based DD aims to learn a series of low-dimensional codes for generating highly informative images through a specific generator. In [33], the vectors that are put into the GAN generator are learned to produce informative images. Concretely, these investigators inverse real examples with a GAN generator and collect corresponding latent codes. Then, the latent vectors are further optimized with the distribution matching algorithm. By this, the optimized latent vectors can induce more informative synthetic examples with the pretrained GAN generator. In addition to using the GAN, Kim et al. [26] employed a deterministic multi-formation function $\texttt{multi-form}(\cdot)$ as the decoder to create synthetic data from fewer condensed data, i.e., the synthetic data are generated by $\mathcal{S} = \texttt{multi-form}(\mathcal{C})$. Then the condensed data $\mathcal{C}$ is optimized in an end-to-end fashion by the gradient matching approach. Besides, the Partitioning and Expansion augmentation strategy adopted in [34] can also be regarded as a non-parametric decoder to generate more synthetic data based on the distilled codes. Concretely, while the distilled codes possess the same dimension as real images, Zhao et al. [34] first partition each code into several patches and then expand the small patches into the standard dimension during both dataset distillation and network training periods. Recently, Cazenavette et al. [53] argued that this low cross-architecture transferability comes from direct optimization on the pixel space, which makes the distilled data overfit to specific architectures used in the DD process. With this argument, they regularized the distillation by optimizing the intermediate latent codes that are fed into pretrained generative models for synthetic data generation, which largely increases the cross-architecture performance due to the more realistic synthetic data.

Different from code-based DD, decoder-based DD solely learns decoders that are used to produce highly informative data. In [17], a generative teaching network (GTN) that employs a trainable network was proposed to generate synthetic images from random noise based on given labels, and the meta-gradients are backpropagated via BPTT to update the GTN rather than the synthetic data.

Naturally, code-decoder DD combines code-based and decoder-based DD and allows the training on both codes and decoders. Deng and Russakovsky [18] generated synthetic images via the matrix multiplication between codes and decodes, which they call the *memory* and *addressing function*. Specifically, they use the memory $\mathcal{M} = \{\boldsymbol{b}_1, \ldots, \boldsymbol{b}_K\}$ to store the bases $\boldsymbol{b}_i \in \mathbb{R}^d$ that have the same dimension as target data; and r addressing functions of $\mathcal{A} = \{\boldsymbol{A}_1, \ldots, \boldsymbol{A}_r\}$ are used to recover the synthetic images according to the given one-hot label $\boldsymbol{y}$ as follows:

$$\boldsymbol{s}_i^\top = \boldsymbol{y}^T \boldsymbol{A}_i [\boldsymbol{b}_1, \ldots, \boldsymbol{b}_K]^\top, \tag{37}$$

where the one-hot label $\boldsymbol{y} \in \mathbb{R}^{C\times 1}$, and $\boldsymbol{A}_i \in \mathbb{R}^{C\times K}$, and C is the number of classes. By plugging different addressing functions $\boldsymbol{A}_i$ into (37), a total number of r synthetic images can be generated from the bases for each category. During the distillation process, these two elements of $\mathcal{M}$ and $\mathcal{A}$ were optimized with the meta-learning framework. Different from matrix multiplication in [18], Liu et al. [30] employ *hallucinator* networks as decoders to generate synthetic data from *basis*. Specifically, a hallucinator network consists of three parts of an encoder $\texttt{enc}$, an affine transformation with trainable scale σ and shift μ, and a decoder $\texttt{dec}$, then a basis $\boldsymbol{b}$ is put into the hallucinator network to generate corresponding synthetic image $\boldsymbol{s}$ as follows:

$$\mathbf{f}_1 = \texttt{enc}(\boldsymbol{b}), \quad \mathbf{f}_2 = \sigma \times \mathbf{f}_1 + \mu, \quad \boldsymbol{s} = \texttt{dec}(\mathbf{f}_2), \tag{38}$$

where the multiplication is element-wise. In addition, to enlarge the knowledge divergence encoded by different hallucinator networks for efficient compression, these researchers proposed an adversarial contrastive constraint that maximizes the divergence of features of different generated synthetic data. A similar code-decoder factorization method is also presented in Lee et al. [35], where they adopted an improved distribution matching to optimize the latent codes and decoders. Recently, Zhang et al. [54] employ the powerful generative model as the decoder and optimize it and corresponding latent codes. Specifically, each low-dimensional latent code can generate synthetic images with different classes by feeding the generative model with corresponding conditional class embedding. By this, the number of learnable parameters is not affected by both the real image resolution and the number of classes, which is promising for tackling the large-scale distillation problem.

Through implicitly learning the latent codes and decoders, factorized DD possesses advantages such as more compact representation and shared representation across classes that consequently improve the dataset distillation performance. Notably, this code-decoder factorization is compatible with the aforementioned distillation approaches. Therefore, the exploration of synthetic data generation and distillation frameworks can promote dataset distillation in parallel.

VI. Performance Comparison

To demonstrate the effectiveness of dataset distillation, we collect and summarize the classification performance of some characteristic dataset distillation approaches on the following image datasets: MNIST [56], FashionMNIST [57], SVHN [58], CIFAR-10/100 [59], Tiny ImageNet [60], and ImageNet-1K [49]. The details of these datasets are presented as follows. Recently, Cui et al. [61] publish a benchmark in terms of dataset distillation, however, it only contains five DD methods, and we provide a comprehensive comparison of over 15 existing dataset distillation methods in this survey.

MNIST is a black-and-white dataset that consists of 60,000 training images and 10,000 test images from 10 different classes; the size of each example image is 28×28 pixels. SVHN is a colorful dataset and consists of 73,257 digits and 26,032 digits for training and testing, respectively, and the example images in SVHN are 32×32 RGB images. CIFAR-10/100 are composed of 50,000 training images and 10,000 test images from 10 and 100 different classes, respectively. The RGB images in CIFAR-10/100 are comprised of 32×32 pixels. Tiny ImageNet consists of 100,000 and 10,000 64×64 RGB images from 200 different classes for training and testing, respectively. ImageNet-1K is a large image dataset that consists of over 1,000,000 high-resolution RGB images from 1,000 different classes.

An important factor affecting the test accuracy *w.r.t.* the distilled dataset is the *distillation budget*, which constrains the size of the distilled dataset by the notion of images allocated per class (IPC). Usually, the distilled dataset is set to have the same number of classes as the target dataset. Therefore, for a target dataset with 100 classes, setting the distillation budget to IPC $= 10$ suggests that there are a total of $10 \times 100 = 1,000$ images in the distilled dataset.

A. Standard Benchmark

For a comprehensive comparison, we also present the test accuracy *w.r.t.* a random select search, coreset approach, and the original target dataset. For the coreset approach, the Herding algorithm [62] is employed in this survey due to its superior performance. Notably, for most parametric DD methods, ConvNet [24], [63], which consists of three convolutional blocks, is the default architecture to distill the synthetic data; and all the test accuracy of distilled data are obtained by training the same architecture of ConvNet for a fair comparison.

The empirical comparison results associated with MNIST, FashionMNIST, SVHN, CIFAR-10/100, and Tiny ImageNet are presented in Table I. In spite of severe scale issues and few implementations on ImageNet-1K, we present the DD results of ImageNet-1K in Table II for completion. As shown in these tables, many methods are not tested on some datasets due to the lack of meaning for easy datasets or scalability problems for large datasets. We preserve these blanks in tables for a more fair and comprehensive comparison. To make the comparison clear, we use orange numbers for the best of two nonfactorized DD methods and blue numbers for the best factorized DD methods in each category in Table I. In addition, we note the factorized DD methods with *. Based on the performance comparison in the tables, we observe the following:

TABLE I
PERFORMANCE COMPARISON OF DIFFERENT DATASET DISTILLATION METHODS ON DIFFERENT DATASETS

Methods	Schemes	MNIST			FashionMNIST			SVHN			CIFAR-10			CIFAR-100			Tiny ImageNet		
		1	10	50	1	10	50	1	10	50	1	10	50	1	10	50	1	10	50
Random	-	64.9 ±3.5	95.1 ±0.9	97.9 ±0.2	51.4 ±3.8	73.8 ±0.7	82.5 ±0.7	14.6 ±1.6	35.1 ±4.1	70.9 ±0.9	14.4 ±2.0	26.0 ±1.2	43.4 ±1.0	4.2 ±0.3	14.6 ±0.5	30.0 ±0.4	1.4 ±0.1	5.0 ±0.2	15.0 ±0.4
Herding	-	89.2 ±1.6	93.7 ±0.3	94.8 ±0.2	67.0 ±1.9	71.1 ±0.7	71.9 ±0.8	20.9 ±1.3	50.5 ±3.3	72.6 ±0.8	21.5 ±1.2	31.6 ±0.7	40.4 ±0.6	8.4 ±0.3	17.3 ±0.3	33.7 ±0.5	2.8 ±0.2	6.3 ±0.2	16.7 ±0.3
DD [11]	BPTT	-	79.5 ±8.1	-	-	-	-	-	-	-	-	36.8 ±1.2	-	-	-	-	-	-	-
LD [15]	BPTT	60.9 ±3.2	87.3 ±0.7	93.3 ±0.3	-	-	-	-	-	-	25.7 ±0.7	38.3 ±0.4	42.5 ±0.4	11.5 ±0.4	-	-	-	-	-
DC [24]	GM	91.7 ±0.5	94.7 ±0.2	98.8 ±0.2	70.5 ±0.6	82.3 ±0.4	83.6 ±0.4	31.2 ±1.4	76.1 ±0.6	82.3 ±0.3	28.3 ±0.5	44.9 ±0.5	53.9 ±0.5	12.8 ±0.3	26.6 ±0.3	32.1 ±0.3	-	-	-
DSA [48]	GM	88.7 ±0.6	97.8 ±0.1	**99.2 ±0.1**	70.6 ±0.6	86.6 ±0.3	88.7 ±0.2	27.5 ±1.4	**79.2 ±0.5**	84.4 ±0.4	28.8 ±0.7	52.1 ±0.5	60.6 ±0.5	13.9 ±0.4	32.4 ±0.3	38.6 ±0.3	-	-	-
DCC [25]	GM	-	-	-	-	-	-	47.5 ±2.6	**80.5 ±0.6**	**87.2 ±0.3**	34.0 ±0.7	54.5 ±0.5	64.2 ±0.4	14.6 ±0.3	33.5 ±0.3	39.3 ±0.4	-	-	-
MTT [27]	TM	91.4 ±0.9	97.3 ±0.1	98.5 ±0.1	75.1 ±0.9	**87.2 ±0.3**	88.3 ±0.1	-	-	-	46.3 ±0.8	65.3 ±0.7	71.6 ±0.2	24.3 ±0.3	40.1 ±0.4	47.7 ±0.2	8.8 ±0.3	23.2 ±0.2	**28.0 ±0.3**
FTD [28]	TM	-	-	-	-	-	-	-	-	-	46.8 ±0.3	**66.6 ±0.3**	**73.8 ±0.2**	25.2 ±0.2	**43.4 ±0.3**	**50.7 ±0.3**	10.4 ±0.3	24.5 ±0.2	-
TESLA [29]	TM	-	-	-	-	-	-	-	-	-	48.5 ±0.8	66.4 ±0.8	72.6 ±0.7	24.8 ±0.4	41.7 ±0.3	**47.9 ±0.3**	7.7 ±0.2	18.8 ±1.3	**27.9 ±1.2**
DM [31]	DM	89.2 ±1.6	97.3 ±0.3	94.8 ±0.2	-	-	-	-	-	-	26.0 ±0.8	48.9 ±0.6	63.0 ±0.4	11.4 ±0.3	29.7 ±0.3	43.6 ±0.4	3.9 ±0.2	12.9 ±0.4	24.1 ±0.3
CAFE [32]	DM	90.8 ±0.5	97.5 ±0.1	98.9 ±0.2	73.7 ±0.7	83.0 ±0.3	88.2 ±0.3	42.9 ±3.0	77.9 ±0.6	82.3 ±0.4	31.6 ±0.8	50.9 ±0.5	62.3 ±0.4	14.0 ±0.3	31.5 ±0.2	42.9 ±0.2	-	-	-
KIP [19, 20]	KRR	90.1 ±0.1	97.5 ±0.0	98.3 ±0.1	73.5 ±0.5	86.8 ±0.1	88.0 ±0.1	**57.3 ±0.1**	75.0 ±0.1	**85.0 ±0.1**	49.9 ±0.2	62.7 ±0.3	68.6 ±0.2	15.7 ±0.2	28.3 ±0.1	-	-	-	-
FRePo [21]	KRR	93.0 ±0.4	**98.6 ±0.1**	**99.2 ±0.0**	75.6 ±0.3	86.2 ±0.2	**89.6 ±0.1**	-	-	-	46.8 ±0.7	65.5 ±0.4	71.7 ±0.2	**28.7 ±0.1**	42.5 ±0.2	44.3 ±0.2	**15.4 ±0.3**	**25.4 ±0.2**	-
RFAD [22]	KRR	**94.4 ±1.5**	98.5 ±0.1	98.8 ±0.1	**78.6 ±1.3**	87.0 ±0.5	88.8 ±0.4	**52.2 ±2.2**	74.9 ±0.4	80.9 ±0.3	**53.6 ±1.2**	66.3 ±0.5	71.1 ±0.4	26.3 ±1.1	33.0 ±0.3	-	-	-	-
RCIG [23]	KRR	**94.7 ±0.5**	**98.9 ±0.0**	**99.2 ±0.0**	**79.8 ±1.1**	**88.5 ±0.2**	**90.2 ±0.2**	-	-	-	**53.9 ±1.0**	**69.1 ±0.4**	**73.5 ±0.3**	**39.3 ±0.4**	**44.1 ±0.4**	46.7 ±0.3	**25.6 ±0.3**	**29.4 ±0.2**	-
IDC* [26]	GM	94.2 ±0.2	98.4 ±0.1	99.1 ±0.1	81.0 ±0.2	86.0 ±0.3	86.2 ±0.2	68.1 ±0.1	87.3 ±0.2	90.2 ±0.1	50.0 ±0.4	67.5 ±0.5	74.5 ±0.1	-	44.8 ±0.2	-	-	-	-
DREAM* [55]	GM	95.7 ±0.3	98.6 ±0.1	99.2 ±0.1	81.3 ±0.2	86.4 ±0.3	86.8 ±0.3	69.8 ±0.8	87.9 ±0.4	90.5 ±0.1	51.1 ±0.3	69.4 ±0.4	74.8 ±0.1	29.5 ±0.3	46.8 ±0.7	**52.6 ±0.4**	10.0 ±0.4	23.9 ±0.4	**29.5 ±0.3**
RTP* [18]	BPTT	**98.7 ±0.7**	**99.3 ±0.5**	**99.4 ±0.4**	**88.5 ±0.1**	**90.0 ±0.7**	**91.2 ±0.3**	**87.3 ±0.1**	89.1 ±0.2	89.5 ±0.2	**66.4 ±0.4**	71.2 ±0.4	73.6 ±0.4	34.0 ±0.4	42.9 ±0.7	-	16.0 ±0.7	-	-
HaBa* [30]	TM	-	-	-	-	-	-	69.8 ±1.3	83.2 ±0.4	88.3 ±0.1	48.3 ±0.8	69.9 ±0.4	74.0 ±0.2	33.4 ±0.4	40.2 ±0.2	47.0 ±0.2	-	-	-
IDM* [34]	DM	-	-	-	-	-	-	-	-	-	45.6 ±0.7	58.6 ±0.1	67.5 ±0.1	20.1 ±0.3	45.1 ±0.1	50.0 ±0.2	10.1 ±0.2	21.9 ±0.2	27.7 ±0.3
KFS* [35]	DM	-	-	-	-	-	-	82.9 ±0.4	**91.4 ±0.2**	**92.2 ±0.1**	59.8 ±0.5	**72.0 ±0.3**	**75.0 ±0.2**	**40.0 ±0.5**	**50.6 ±0.2**	-	**22.7 ±0.3**	**27.8 ±0.2**	-
Whole	-	99.6 ±0.0			93.5 ±0.1			95.4 ±0.1			84.8 ±0.1			56.2 ±0.3			37.6 ±0.4		

Abbreviations of GM, TM, and DM are for gradient matching, trajectory matching, and distribution matching, respectively. Whole denotes the test accuracy in terms of the whole target dataset. The factorized DD methods are marked by * .

TABLE II
PERFORMANCE COMPARISON OF DIFFERENT DATASET DISTILLATION METHODS ON IMAGENET-1K

Methods	IPC=1	IPC = 2	IPC = 10	IPC = 50
Random	0.5 ±0.1	0.9 ±0.1	3.6 ±0.1	15.3 ±2.3
DM [31]	1.5 ±0.1	1.7 ±0.1	-	-
FRePo [21]	7.5 ±0.3	9.7 ±0.2	-	-
TESLA [29]	7.7 ±0.2	10.5 ±0.3	**17.8 ±1.3**	**27.9 ±1.2**
RCIG [23]	**15.6 ±0.2**	**16.6 ±0.1**	-	-
Whole	33.8 ±0.3			

- Dataset distillation can be realized on many datasets with various sizes.
- Dataset distillation methods outperform random pick and coreset selection by large margins.
- KRR and trajectory matching approaches possess advanced DD performance *w.r.t.* three larger datasets of CIFAR-10/100 and Tiny ImageNet among nonfactorized DD methods.
- Factorized dataset distillation by optimizing the latent codes and decoders can largely improve the test accuracy of distilled data.
- KFS [35] has the best overall performance among different datasets of CIFAR-10/100 and Tiny ImageNet.

B. Cross-Architecture Transferability

The standard benchmark in Table I only shows the performance of DD methods in terms of the specific ConvNet, while the distilled dataset is usually used to train other unseen network architectures. Therefore, measuring the DD performance on different architectures is also important for a more comprehensive evaluation. However, there is not a standard set of architectures to evaluate the cross-architecture transferability of DD algorithms, and the choice of architectures is highly different in the DD literature.

In this survey, we collect synthetic data distilled by different methods on ConvNet to train other five popular architectures of ResNet-10/18/152 [2], DenseNet-121 [64], and Vision Transformer (ViT)[1] [65] to evaluate the cross-architecture transferability of distilled data, as shown in Table III. By investigating these tables, we can obtain several observations as follows:

- There is a significant performance drop for nonfactorized DD methods when the distilled data are used to train unseen architectures.
- The distilled data do not always possess the best performance across different architectures, which underlines the importance of evaluation on multiple architectures.

[1]The ViT architecture has 4×4 patch resolution, 6 layers, 8 heads, hidden dimension of 512, and the MLP dimension of 512.

TABLE III
CROSS-ARCHITECTURE PERFORMANCE *W.R.T.* RESNET (RN), DENSENET (DN), AND VIT ON CIFAR-10

Methods	IPC=1						IPC=10						IPC=50					
	ConvNet	RN-10	RN-18	RN-152	DN-121	ViT	ConvNet	RN-10	RN-18	RN-152	DN-121	ViT	ConvNet	RN-10	RN-18	RN-152	DN-121	ViT
DC [24]	28.3 ±0.5	-	25.6 ±0.6	**15.3 ±0.4**	-	**24.68 ±0.7**	44.9 ±0.5	-	42.1 ±0.6	16.1 ±1.0	-	28.96 ±0.6	53.9 ±0.5	-	45.9 ±1.4	19.7 ±1.2	-	27.61 ±0.3
DSA [48]	28.8 ±0.5	25.1 ±0.8	25.6 ±0.6	15.1 ±0.7	25.9 ±1.8	23.69 ±0.8	52.1 ±0.5	31.4 ±0.9	42.1 ±0.6	16.1 ±1.0	32.9 ±1.0	31.9 ±0.4	60.6 ±0.5	49.0 ±0.7	49.5 ±0.7	20.0 ±1.2	53.4 ±0.8	43.19 ±0.7
MTT [27]	46.3 ±0.8	-	**34.2 ±1.4**	13.4 ±0.9	-	21.67 ±0.5	65.3 ±0.7	-	38.8 ±0.7	15.9 ±0.2	-	34.6 ±0.6	71.6 ±0.2	-	60.0 ±0.7	20.9 ±1.6	-	**48.00 ±0.3**
TESLA [29]	-	-	-	-	-	-	66.4 ±0.8	-	**48.9 ±2.2**	-	-	**34.8 ±1.2**	-	-	-	-	-	-
DM [31]	26.0 ±0.8	13.7 ±1.6	20.6 ±0.5	14.1 ±0.6	12.9 ±1.8	21.34 ±0.2	48.9 ±0.6	31.7 ±1.1	38.2 ±1.1	15.6 ±1.5	32.2 ±0.8	34.4 ±0.5	63.0 ±0.4	49.1 ±0.7	52.8 ±0.4	**21.7 ±1.3**	53.7 ±0.7	45.07 ±0.5
KIP [20]	49.9 ±0.2	-	27.6 ±1.1	14.2 ±0.8	-	16.80 ±0.8	62.7 ±0.3	-	45.2 ±1.4	**16.6 ±1.4**	-	15.9 ±1.1	68.6 ±0.2	-	60.0 ±0.7	20.9 ±1.6	-	18.56 ±0.84
IDC* [26]	50.0 ±0.4	41.9 ±0.6	-	-	39.8 ±1.2	-	67.5 ±0.5	63.5 ±0.1	-	-	61.6 ±0.6	-	74.5 ±0.1	72.4 ±0.5	-	-	71.8 ±0.6	-
KFS* [35]	**59.8 ±0.5**	**47.0 ±0.8**	-	-	**49.5 ±1.3**	-	**72.0 ±0.3**	**70.3 ±0.3**	-	-	**71.4 ±0.4**	-	**75.0 ±0.2**	**75.1 ±0.3**	-	-	**76.3 ±0.4**	-

The factorized DD methods are marked by * .

- The factorized DD methods (IDC and KFS) possess better cross-architecture transferability, i.e., suffer a smaller accuracy drop when encountering unseen architectures.

VII. DATA MODALITIES

Apart from image data that the majority of DD works focus on, some works leverage dataset distillation on multimodal data such as graph, text, *etc.*

A. Graph Data

Regarding a graph dataset of $\mathcal{T} = \{\mathbf{A}, \mathbf{X}, \mathbf{Y}\}$ with N nodes, where $\mathbf{A}$, $\mathbf{X}$, and $\mathbf{Y}$ are the adjacency matrix, the node feature matrix, and the node labels, respectively, a graph neural network $\mathcal{G}$ is trained on $\mathcal{T}$ for node classification. Jin et al. [66] adopted the gradient matching approach to distill the graph $\mathcal{T}$ to a small synthetic graph $\mathcal{S} = \{\mathbf{A}', \mathbf{X}', \mathbf{Y}'\}$ with N' nodes and $N' \ll N$.

However, the number of parameters in the adjacency matrix $\mathbf{A}'$ grows quadratically as N' increases, which greatly impedes the scalability of graph distillation. Considering the implicit correlation between the graph structure and node features, the authors employ an MLP-based model to predict the graph structure based on node features

$$\mathbf{A}'_{ij} = \sigma\left(\frac{\mathrm{MLP}_\Phi\left(\left[\mathbf{x}'_i; \mathbf{x}'_j\right]\right) + \mathrm{MLP}_\Phi\left(\left[\mathbf{x}'_j; \mathbf{x}'_i\right]\right)}{2}\right), \quad (39)$$

where $\sigma(\cdot)$ is the sigmoid function, MLP_Φ is parameterized with Φ, and $[\cdot;\cdot]$ denotes concatenation. With fixing $\mathbf{Y}'$ to ease the difficulty of optimization, the loss function $\mathcal{L}(\mathbf{X}', \Phi)$ of graph distillation for node classification is defined as

$$\sum_{t=0}^{T-1} D\left(\nabla_{\boldsymbol{\theta}} \mathcal{R}\left(\mathcal{G}_{\boldsymbol{\theta}_t}\left(\Phi\left(\mathbf{X}'\right), \mathbf{X}'\right), \mathbf{Y}'\right), \nabla_{\boldsymbol{\theta}} \mathcal{R}\left(\mathcal{G}_{\boldsymbol{\theta}_t}(\mathbf{A}, \mathbf{X}), \mathbf{Y}\right)\right). \quad (40)$$

Moreover, to bypass second-order meta-gradient computation, Liu et al. [30] accelerated the graph distillation through distribution matching from the perspective of receptive fields, which present the local graph *w.r.t.* a specific node.

However, for the graph classification problem where the adjacency matrix is discrete, the conventional DD algorithms are no longer applicable. To distill synthetic graphs with discrete adjacency matrix $\mathbf{A}' \in \{0,1\}^{|\mathbf{A}'|}$, Jin et al. [67] formulate $\mathbf{A}'$ as a probabilistic graph with Bernoulli distribution

$$\mathbf{P}_{\Omega_{ij}}\left(\mathbf{A}'_{ij}\right) = \mathbf{A}'_{ij}\sigma\left(\Omega_{ij}\right) + \left(1 - \mathbf{A}'_{ij}\right)\sigma\left(-\Omega_{ij}\right), \quad (41)$$

where σ is the sigmoid function and $\Omega_{ij} \in \mathbb{R}$ is the learnable parameter. Then with the reparameterization trick, the parameter Ω becomes differentiable and $\mathbf{A}'$ is accordingly synthesized by optimizing Ω.

B. Text Data

For the discrete text data, Li and Li [68] converted each piece of text into a continuous embedding matrix to adjust DD algorithms, and then a few high-informative synthetic embedding matrices were distilled for text classification. To fine-tune large language models like BERT [69] with the distilled text data, Maekawa et al. [70] further optimized the soft label by decreasing the KL divergence between the attention probabilities of distilled data and the model during the DD process, which facilitates efficient knowledge transfer to transformer models.

VIII. APPLICATION

Due to this superior performance in compressing massive datasets, dataset distillation has been widely employed in many application domains that limit training efficiency and storage, including continual learning and neural architecture search. Furthermore, due to the correspondence between examples and gradients, dataset distillation can also benefit privacy preservation, federated learning, and adversarial robustness. In this section, we briefly review these applications *w.r.t* dataset distillation.

A. Continual Learning

During to the training process, when there is a shift in the training data distribution, the model will suddenly lose its ability to predict the previous data distribution. This phenomenon is referred to as *catastrophic forgetting* and is common in deep learning. To overcome this problem, continual learning has been developed to incrementally learn new tasks while preserving performance on old tasks [62], [71]. A common method used in continual learning is the replay-based strategy, which allows a limited memory to store a few training examples for rehearsal in the following training. Therefore, the key to the replay-based strategy is to select highly informative training examples to store. Benefiting from extracting the essence of datasets, the dataset distillation technique has been employed to compress data for memory with limited storage [24], [72]. Because the incoming data have a changing distribution, the frequency is high for updating the elements in memory, which leads to strict

requirements for the efficiency of dataset distillation algorithms. To conveniently embed dataset distillation into the replay-based strategy, Wiewel and Yang [73] and Sangermano et al. [74] decomposed the process of synthetic data generation by linear or nonlinear combination, and thus fewer parameters were optimized during dataset distillation. Gu et al. [75] develop the dynamic memory to maintain both distilled and real data, which helps better summarize data information with the distilled data.

In addition to enhancing replay-based methods, dataset distillation can also learn a sequence of stable datasets, and the network trained on these stable datasets will not suffer from catastrophic forgetting [76].

B. Neural Architecture Search

For a given dataset, the technique of neural architecture search (NAS) aims to find an optimal architecture from thousands of network candidates for better generalization. The NAS process usually includes training the network candidates on a small proxy of the original dataset to save training time, and the generalization ranking can be estimated according to these trained network candidates. Therefore, it is important to design the proxy dataset such that models trained on it can reflect the model's true performance in terms of the original data, but the size of the proxy dataset requires control for the sake of efficiency. To construct proxy datasets, conventional methods have been developed, including random selection or greedy search, without altering the original data [77], [78]. Such et al. [17] first proposed optimizing a highly informative dataset as the proxy for network candidate selection. More subsequent works have considered NAS as an auxiliary task for testing the proposed dataset distillation algorithms [24], [28], [31], [48], [61]. Through simulation on the synthetic dataset, a fairly accurate generalization ranking can be collected for selecting the optimal architecture while considerably reducing the training time.

C. Federated Learning

Federated learning has received increasing attention during training neural networks in the past few years due to its advantages in distributed training and private data protection [79]. The federated learning framework consists of multiple clients and one central server, and each client possesses exclusive data for training the corresponding local model [80]. In one round of federated learning, clients transmit the induced gradients or model parameters to the server after training with their exclusive data. Then the central server aggregates the received gradients or parameters to update the model parameters and broadcast new parameters to clients for the next round. In the federated learning scenario, the data distributed in different clients are often non-i.i.d data, which causes a biased minimum *w.r.t.* the local model and significantly hinders the convergence speed. Consequently, the data heterogeneity remarkably imposes a communication burden between the server and clients. Goetz and Tewari [81], Wang et al. [82] proposed distilling a small set of synthetic data from original exclusive data by matching gradients, and then the synthetic data instead of a large number of gradients were transmitted to the server for model updating. Other distillation approaches such as BPTT [83], [84], KRR [85], and distribution matching [86], were also employed to compress the exclusive data to alleviate the communication cost at each transition round. However, Liu et al. [87] discovered that synthetic data generated by dataset distillation are still heterogeneous. To address the problem, these researchers proposed two strategies of dynamic weight assignment and meta knowledge sharing during the distillation process, which significantly accelerate the convergence speed of federated learning. Apart from compressing the local data, Pi et al. [88] also distilled data via trajectory matching in the server, which allows the synthetic data to possess global information. Then the distilled data can be used to fine-tune the server model for convergence speed up.

D. Other Applications

Apart from the aforementioned applications, we also summarize other applications in terms of dataset distillation as follows.

Because of their small size, synthetic datasets have been applied to explainability algorithms [22]. In particular, due to the small size of synthetic data, it is easy to measure how the synthetic examples influence the prediction of test examples. If the test and training images both rely on the same synthetic images, then the training image will greatly influence the prediction of the test image. In other words, the synthetic set becomes a bridge to connect the training and testing examples.

Leveraging the characteristic of capturing the essence of datasets, Cazenavette et al. [89] and Chen et al. [90] adopted dataset distillation for visual design. Specifically, Cazenavette et al. [89] generated the representative textures by randomly cropping the synthetic images during the distillation process. In addition to extracting texture, Chen et al. [90] imposed the synthetic images to model the outfit compatibility through dataset distillation.

Due to their small size and abstract visual information, distilled data can also be applied in medicine, especially in medical image sharing [91]. Empirical studies on gastric X-ray images have shown the advantages of DD in medical image transition and the anonymization of patient information [92], [93]. Through dataset distillation, hospitals can share their valuable medical data at lower cost and risk to collaboratively build powerful computer-aided diagnosis systems.

IX. Challenges and Future Directions

Due to its superiority in compressing datasets and training speedup, dataset distillation has promising prospects in a wide range of areas. In the following, we discuss existing challenges in terms of dataset distillation. Furthermore, we investigate plausible directions and provide insights into dataset distillation to promote future studies.

A. Challenges

Scalability: Although many approaches have been proposed to decrease memory utilization [29] and accelerate convergence [55] for efficient DD, the scalability of DD is still a serious issue due to the hardness of optimizing numerous

hyperparameters (e.g., pixels for image data). For example, when distilling the 1000-class ImageNet with the input resolution of 224×224 and IPC of 50, totally $224 \times 224 \times 1000 \times 50 \approx 2.5$billon hyperparameters (pixels) are required to simultaneously optimize. Therefore, how to effectively search a feasible solution over the extremely high-dimensional hyperparameter space is important to scale DD algorithms to more complicated data (e.g., image data with higher resolution, 3D data).

Multimodal Dataset Distillation: Whereas DD algorithms have achieved remarkable progress in terms of image [11], [24], graph [66], text [68], tabular [94], and recommendation system data [95], their applications on other data modalities are still underexplored. Especially for the modalities such as video that rich spatial and temporal information are tangled with each other in a single data [96], it is still challenging for DD algorithms to simultaneously distill the spatial and temporal features into a small volume of data.

DD With Complex Labels: The majority of DD techniques lie in the single-label classification task where its label space is an integer set. As for other challenging tasks such as detection [97] or segmentation [98], their label spaces are located in much higher dimensional spaces. Unlike DD with single-label classification that the specific number of synthetic images (IPC) are pre-assigned for each class, enumerated assignment is impractical when the label space is highly dimensional. Therefore, to represent the rich label space, high-dimensional labels of synthetic data require cautious adjusting, which significantly hinders the DD efficiency especially when jointly optimized with the input data.

Theory of Dataset Distillation: Although various DD algorithms have been emerging, there have been few investigations on discussing the theory behind dataset distillation. Nevertheless, developing the theory is extremely necessary and will considerably promote the dataset distillation by directly improving the performance and also suggesting the correct direction of development. For example, the theory can help derive a better definition of the dataset knowledge and consequently increase the performance. In addition, the theoretical correlation between the distillation budget (the size of synthetic datasets) and performance can provide an upper bound on the test error, which can offer a holistic understanding and prevent researchers from blindly improving the DD performance. Hence, a solid theory is indispensable to take the development of DD to the next stage.

B. Future Directions

Despite aforementioned challenges existing in DD, there are still many promising directions that are worth exploring to (1) enhance dataset distillation performance and (2) develop trustworthy dataset distillation.

Optimization Objective of DD: As discussed in Sections V and VI-A, optimizing factorized codes and decoders that cooperatively generate synthetic data outperform the direct optimization by a large margin. This suggests that the selection of optimization objectives (e.g., RGB pixels, latent codes and decoders) has a significant influence on the final distillation performance. Apart from optimization on original data or factorized spaces, Wang et al. [99] turned to optimize the generative model to synthesize unlimited informative data, and Cazenavette et al. [53] optimized the intermediate features of generative models to generate more realistic synthetic data for better cross-architecture generalization. Therefore, investigating optimization on proper information carriers, such as optimizing a few 3D data to summarize information of 2D images, is a promising direction for further DD performance improvement.

Model Augmentation: During the process of DD, neural networks within different training stages are involved to compute the distillation loss, and then the synthetic data are accordingly updated. This triggers a problem that distilled data are sensitive to network parameters, and causes unsatisfied DD performance when training new networks whose parameters have a large discrepancy to models used for calculating distillation loss. Therefore, the sensitivity to model parameters is closely related to DD performance. To alleviate this sensitivity, early works use a simple model augmentation strategy that conducts DD on networks with different initialization and training stages [11], [24]. Recently, Zhang et al. [100] showed that employing early-stage models trained with a few epochs as the initialization can achieve better distillation performance, and they further proposed weight perturbation methods to efficiently generate early-stage models for calculating distillation loss. Therefore, it is important for dataset distillation to investigate model augmentation algorithms to reduce the sensitivity of distilled data to model parameters and further improve DD performance.

Cross-Architecture Transferability: Because the distillation of synthetic data is closely related to the specific network architecture (or kernels in the KRR approach), there is naturally a performance drop when distilled datasets are applied to train networks with other unseen architectures. Recently, Cazenavette et al. [53] argued that this low cross-architecture transferability comes from direct optimization on the pixel space, which makes the distilled data overfit to specific architectures used in the DD process. With this argument, they regularized the distillation by optimizing the latent codes that are fed into pretrained generative models for distilled data generation, which largely increases the cross-architecture performance due to more realistic synthetic data. A decent cross-architecture generalization is significant and allows synthetic data to train more complex models that are prohibitively used in DD process due to unbearable computing overhead.

Private Dataset Distillation: As the real data are involved in dataset distillation, it is necessary to analyze the distilled data from the perspective of privacy protection.

Theoretically, Dong et al. [101] and Zheng and Li [102] built the connection between dataset distillation and differential privacy (DP), and proved the superiority of dataset distillation in privacy preservation over conventional private data generation methods. However, Carlini et al. [103] argued that there exist flaws in both experimental and theoretical evidence in [101]: 1) membership inference baseline should include all training examples, while only 1% of training points are used to match the synthetic sample size, which thus contributes to the high attack accuracy of baseline; and 2) the privacy analysis is based on an assumption stating that the output of learning algorithms

follows a exponential distribution in terms of loss, while the assumption has already implied differential privacy. Empirically, Chen et al. [104] employed gradient matching to distill private datasets by adding DP noise to the gradients of synthetic data, which was followed by Vinaroz and Park [105] who privatized KIP with DP-SGD to achieve better privacy-accuracy trade-off compared to [104]. Therefore, a powerful private dataset distillation algorithm helps reassuringly publish distilled data and alleviate data abuse.

Robust Dataset Distillation: It is significant to develop robust DD algorithms such that models trained on distilled data own satisfied robustness *w.r.t.* malicious attacks and ambiguous test data, or on the contrary, attack downstream models via manipulating the synthetic data.

To enhance adversarial robustness, Tsilivis et al. [106] and Wu et al. [107] employed dataset distillation to extract the information of adversarial examples and generate robust datasets. Then standard network training on the distilled robust dataset is sufficient to achieve satisfactory robustness to adversarial perturbations, which substantially saves computing resources compared to expensive adversarial training. For backdoor attack, Liu et al. [108] considered injecting backdoor triggers into the small distilled dataset. Because the synthetic data possess a small size, direct trigger insertion is less effective and also perceptible. Therefore, these coworkers turned to insert triggers into the target dataset during the dataset distillation procedure. In addition, they propose to iteratively optimize the triggers during the distillation process to preserve the triggers' information for better backdoor attack performance. These defense and attack algorithms in terms of DD invoke the distillation of robust synthetic datasets, and models trained on them can better defend against various attacks.

Recently, Zhu et al. [109] revealed that models trained on distilled data tend to output over-confident predictions and have a bad calibration, due to discarding semantically meaningful information. To tackle this problem, they propose to randomly mask the synthetic data to compel them to contain more semantically complete information during the distillation process. Apart from estimating proper uncertainty for in-distribution data, it is also important to detect out-of-distribution (OoD) examples for models deployed in the open world. Ma et al. [110] additionally synthesized outliers based on corruption transformations of real data, which are called pseudo-outliers. Then the synthetic outliers are encompassed to train downstream models for enhancing the capability of OoD detection. Therefore, it is significant for trustworthy learning to construct distilled datasets that can induce proper uncertainty *w.r.t.* both in-distribution and out-of-distribution test data.

REFERENCES

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[3] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[4] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 1597–1607.

[5] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9729–9738.

[6] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818–2826.

[7] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, inception-ResNet and the impact of residual connections on learning," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 4278–4284.

[8] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.

[9] S. Sagiroglu and D. Sinanc, "Big data: A review," in *Proc. Int. Conf. Collaboration Technol. Syst.*, 2013, pp. 42–47.

[10] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, Florence, Italy:Association for Computational Linguistics, 2019, pp. 3645–3650. [Online]. Available: https://aclanthology.org/P19--1355

[11] T. Wang, J.-Y. Zhu, A. Torralba, and A. A. Efros, "Dataset distillation," 2018, *arXiv:1811.10959*.

[12] J. Nalepa and M. Kawulok, "Selecting training sets for support vector machines: A review," *Artif. Intell. Rev.*, vol. 52, no. 2, pp. 857–900, 2019.

[13] B. Mirzasoleiman, J. Bilmes, and J. Leskovec, "Coresets for data-efficient training of machine learning models," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 6950–6960.

[14] C. Coleman et al., "Selection via proxy: Efficient data selection for deep learning," in *Proc. Int. Conf. Learn. Representations*, 2020. [Online]. Available: https://openreview.net/pdf?id=HJg2b0VYDr

[15] O. Bohdal, Y. Yang, and T. Hospedales, "Flexible dataset distillation: Learn labels instead of images," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020. [Online]. Available: https://meta-learn.github.io/2020/papers/42_paper.pdf

[16] I. Sucholutsky and M. Schonlau, "Soft-label dataset distillation and text dataset distillation," in *Proc. Int. Joint Conf. Neural Netw.*, 2021, pp. 1–8.

[17] F. P. Such, A. Rawal, J. Lehman, K. Stanley, and J. Clune, "Generative teaching networks: Accelerating neural architecture search by learning to generate synthetic training data," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 9206–9216.

[18] Z. Deng and O. Russakovsky, "Remember the past: Distilling datasets into addressable memories for neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022. [Online]. Available: https://openreview.net/pdf?id=RYZyj_wwgfa

[19] T. Nguyen, Z. Chen, and J. Lee, "Dataset meta-learning from kernel ridge-regression," in *Proc. Int. Conf. Learn. Representations*, 2021. [Online]. Available: https://openreview.net/pdf?id=l-PrrQrK0QR

[20] T. Nguyen, R. Novak, L. Xiao, and J. Lee, "Dataset distillation with infinitely wide convolutional networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 5186–5198.

[21] Y. Zhou, E. Nezhadarya, and J. Ba, "Dataset distillation using neural feature regression," in *Proc. Adv. Neural Inf. Process. Syst.*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022. [Online]. Available: https://openreview.net/forum?id=2clwrA2tfik

[22] N. Loo, R. Hasani, A. Amini, and D. Rus, "Efficient dataset distillation using random feature approximation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022. [Online]. Available: https://openreview.net/pdf?id=h8Bd7Gm3muB

[23] N. Loo, R. Hasani, M. Lechner, and D. Rus, "Dataset distillation with convexified implicit gradients," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 22649–22674.

[24] B. Zhao, K. R. Mopuri, and H. Bilen, "Dataset condensation with gradient matching," in *Proc. Int. Conf. Learn. Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=mSAKhLYLSsl

[25] S. Lee, S. Chun, S. Jung, S. Yun, and S. Yoon, "Dataset condensation with contrastive signals," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 12352–12364.

[26] J.-H. Kim et al., "Dataset condensation via efficient synthetic-data parameterization," in *Proc. 39th Int. Conf. Mach. Learn.*, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., PMLR, 2022, pp. 11102–11118. [Online]. Available: https://proceedings.mlr.press/v162/kim22c.html

[27] G. Cazenavette, T. Wang, A. Torralba, A. A. Efros, and J.-Y. Zhu, "Dataset distillation by matching training trajectories," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10708–10717.

[28] J. Du, Y. Jiang, V. T. F. Tan, J. T. Zhou, and H. Li, "Minimizing the accumulated trajectory error to improve dataset distillation," 2022, *arXiv:2211.11004*.

[29] J. Cui, R. Wang, S. Si, and C.-J. Hsieh, "Scaling up dataset distillation to ImageNet-1K with constant memory," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 6565–6590.

[30] S. Liu, K. Wang, X. Yang, J. Ye, and X. Wang, "Dataset distillation via factorization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022. [Online]. Available: https://openreview.net/pdf?id=luGXvawYWJ

[31] B. Zhao and H. Bilen, "Dataset condensation with distribution matching," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2023, pp. 6503–6512.

[32] K. Wang et al., "CAFE: Learning to condense dataset by aligning features," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12196–12205.

[33] B. Zhao and H. Bilen, "Synthesizing informative training samples with GAN," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022. [Online]. Available: https://openreview.net/pdf?id=frAv0jtUMfS

[34] G. Zhao, G. Li, Y. Qin, and Y. Yu, "Improved distribution matching for dataset condensation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 7856–7865.

[35] H. B. Lee, D. B. Lee, and S. J. Hwang, "Dataset condensation with latent space knowledge factorization and sharing," 2022, *arXiv:2208.00719*.

[36] D. Maclaurin, D. Duvenaud, and R. Adams, "Gradient-based hyperparameter optimization through reversible learning," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2015, pp. 2113–2122.

[37] N. Sachdeva and J. McAuley, "Data distillation: A survey," *Trans. Mach. Learn. Res.*, 2023. [Online]. Available: https://openreview.net/forum?id=lmXMXP74TO

[38] T. Hospedales, A. Antoniou, P. Micaelli, and A. Storkey, "Meta-learning in neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 9, pp. 5149–5169, Sep. 2022.

[39] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proc. IEEE*, vol. 78, no. 10, pp. 1550–1560, Oct. 1990.

[40] K. B. Petersen and M. S. Pedersen, "The matrix cookbook," *Techn. Univ. Denmark*, Nov. 2012.

[41] A. Jacot, F. Gabriel, and C. Hongler, "Neural tangent kernel: Convergence and generalization in neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 8580–8589.

[42] J. Platt et al., "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," *Adv. Large Margin Classifiers*, vol. 10, no. 3, pp. 61–74, 1999.

[43] H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein, "Visualizing the loss landscape of neural nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 6391–6401.

[44] E. Golikov, E. Pokonechnyy, and V. Korviakov, "Neural tangent kernel: A survey," 2022, *arXiv:2208.13614*.

[45] A. Bietti and J. Mairal, "On the inductive bias of neural tangent kernels," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, Art. no. 1155.

[46] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 2579–2605, 2008.

[47] Z. Jiang, J. Gu, M. Liu, and D. Z. Pan, "Delving into effective gradient matching for dataset condensation," 2022, *arXiv:2208.00311*.

[48] B. Zhao and H. Bilen, "Dataset condensation with differentiable Siamese augmentation," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 12674–12685.

[49] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, pp. 211–252, 2015.

[50] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *Int. J. Comput. Vis.*, vol. 129, no. 6, pp. 1789–1819, 2021.

[51] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1798–1828, Aug. 2013.

[52] D. Zhang, J. Yin, X. Zhu, and C. Zhang, "Network representation learning: A survey," *IEEE Trans. Big Data*, vol. 6, no. 1, pp. 3–28, Mar. 2020.

[53] G. Cazenavette, T. Wang, A. Torralba, A. A. Efros, and J.-Y. Zhu, "Generalizing dataset distillation via deep generative prior," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 3739–3748.

[54] D. J. Zhang et al., "Dataset condensation via generative model," 2023, *arXiv:2309.07698*.

[55] Y. Liu, J. Gu, K. Wang, Z. Zhu, W. Jiang, and Y. You, "DREAM: Efficient dataset distillation by representative matching," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2023, pp. 17314–17342.

[56] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[57] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.

[58] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng, "Reading digits in natural images with unsupervised feature learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2011. [Online]. Available: https://deeplearningworkshopnips2011.files.wordpress.com/2011/12/12.pdf

[59] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," M.S. thesis, Dept. Comput. Sci., Univ. Toronto, ON, Canada, Citeseer, 2009.

[60] Y. Le and X. Yang, "Tiny ImageNet visual recognition challenge," *CS 231N*, vol. 7, pp. 7, 2015.

[61] J. Cui, R. Wang, S. Si, and C.-J. Hsieh, "DC-BENCH: Dataset condensation benchmark," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022. [Online]. Available: https://openreview.net/pdf?id=Bs8iFQ7AM6

[62] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2001–2010.

[63] S. Gidaris and N. Komodakis, "Dynamic few-shot visual learning without forgetting," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4367–4375.

[64] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4700–4708.

[65] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=YicbFdNTTy

[66] W. Jin, L. Zhao, S. Zhang, Y. Liu, J. Tang, and N. Shah, "Graph condensation for graph neural networks," in *Proc. Int. Conf. Learn. Representations*, 2022. [Online]. Available: https://openreview.net/pdf?id=WLEx3Jo4QaB

[67] W. Jin et al., "Condensing graphs via one-step gradient matching," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 720–730.

[68] Y. Li and W. Li, "Data distillation for text classification," 2021, *arXiv:2104.08448*.

[69] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, Minneapolis, Minnesota:Association for Computational Linguistics, 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19--1423

[70] A. Maekawa, N. Kobayashi, K. Funakoshi, and M. Okumura, "Dataset distillation with attention labels for fine-tuning BERT," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2023, pp. 119–127.

[71] F. M. Castro, M. J. Marín-Jiménez, N. Guil, C. Schmid, and K. Alahari, "End-to-end incremental learning," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 233–248.

[72] A. Carta, A. Cossu, V. Lomonaco, and D. Bacciu, "Distilled replay: Overcoming forgetting through synthetic samples," in *Proc. 1st Int. Workshop Continual Semi-Supervised Learn.*, Springer Nature, 2022, pp. 104–117.

[73] F. Wiewel and B. Yang, "Condensed composite memory continual learning," in *Proc. Int. Joint Conf. Neural Netw.*, 2021, pp. 1–8.

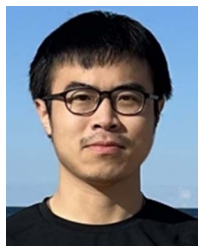
[74] M. Sangermano, A. Carta, A. Cossu, and D. Bacciu, "Sample condensation in online continual learning," in *Proc. Int. Joint Conf. Neural Netw.*, 2022, pp. 01–08.

[75] J. Gu, K. Wang, W. Jiang, and Y. You, "Summarizing stream data for memory-restricted online continual learning," 2023. [Online]. Available: https://arxiv.org/abs/2305.16645

[76] W. Masarczyk and I. Tautkute, "Reducing catastrophic forgetting with learning on synthetic data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2020, pp. 252–253.

[77] V. P. C. C. White, P. Jain, S. Nayak, R. K. Iyer, and G. Ramakrishnan, "Speeding up NAS with adaptive subset selection," in *Proc. 1st Conf. Autom. Mach. Learn.*, 2022. [Online]. Available: https://openreview.net/forum?id=52OEvDa5xrj

[78] G. Li, G. Qian, I. C. Delgadillo, M. Muller, A. Thabet, and B. Ghanem, "SGAS: Sequential greedy architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1620–1630.
[79] Q. Yang, Y. Liu, Y. Cheng, Y. Kang, T. Chen, and H. Yu, "Federated learning," *Synth. Lectures Artif. Intell. Mach. Learn.*, vol. 13, no. 3, pp. 1–207, 2019.
[80] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, PMLR, 2017, pp. 1273–1282.
[81] J. Goetz and A. Tewari, "Federated learning via synthetic data," 2020, *arXiv:2008.04489*.
[82] H.-P. Wang, D. Chen, R. Kerkouche, and M. Fritz, "Fed-GLOSS-DP: Federated, global learning using synthetic sets with record level differential privacy," 2023, *arXiv:2302.01068*.
[83] S. Hu, J. Goetz, K. Malik, H. Zhan, Z. Liu, and Y. Liu, "FedSynth: Gradient compression via synthetic data in federated learning," 2022, *arXiv:2204.01273*.
[84] Y. Zhou, G. Pu, X. Ma, X. Li, and D. Wu, "Distilled one-shot federated learning," 2020, *arXiv:2009.07999*.
[85] R. Song et al., "Federated learning via decentralized dataset distillation in resource-constrained edge environments," 2022, *arXiv:2208.11311*.
[86] Y. Xiong, R. Wang, M. Cheng, F. Yu, and C.-J. Hsieh, "FedDM: Iterative distribution matching for communication-efficient federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 16323–16332.
[87] P. Liu, X. Yu, and J. T. Zhou, "Meta knowledge condensation for federated learning," 2022, *arXiv:2209.14851*.
[88] R. Pi et al., "DYNAFED: Tackling client data heterogeneity with global dynamics," 2022, *arXiv:2211.10878*.
[89] G. Cazenavette, T. Wang, A. Torralba, A. A. Efros, and J.-Y. Zhu, "Wearable ImageNet: Synthesizing tileable textures via dataset distillation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2022, pp. 2278–2282.
[90] Y. Chen, Z. Wu, Z. Shen, and J. Jia, "Learning from designers: Fashion compatibility analysis via dataset distillation," in *Proc. IEEE Int. Conf. Image Process.*, 2022, pp. 856–860.
[91] G. Li, R. Togo, T. Ogawa, and M. Haseyama, "Dataset distillation for medical dataset sharing," in *Proc. AAAI Conf. Artif. Intell. Workshop*, pp. 1–6, 2023.
[92] G. Li, R. Togo, T. Ogawa, and M. Haseyama, "Soft-label anonymous gastric X-ray image distillation," in *Proc. IEEE Int. Conf. Image Process.*, 2020, pp. 305–309.
[93] G. Li, R. Togo, T. Ogawa, and M. Haseyama, "Compressed gastric image generation based on soft-label dataset distillation for medical data sharing," *Comput. Methods Programs Biomed.*, vol. 227, 2022, Art. no. 107189.
[94] D. Medvedev and A. D'yakonov, "New properties of the data distillation method when working with tabular data," in *Proc. Int. Conf. Anal. Images Social Netw. Texts*, 2020, pp. 379–390.
[95] N. Sachdeva, M. Preet Dhaliwal, C.-J. Wu, and J. McAuley, "Infinite recommendation networks: A data-centric approach," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022. [Online]. Available: https://openreview.net/pdf?id=lmmKGi7zXn
[96] S. Herath, M. Harandi, and F. Porikli, "Going deeper into action recognition: A survey," *Image Vis. Comput.*, vol. 60, pp. 4–21, 2017.
[97] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 91–99.
[98] S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz, and D. Terzopoulos, "Image segmentation using deep learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3523–3542, Jul. 2022.
[99] K. Wang, J. Gu, D. Zhou, Z. Zhu, W. Jiang, and Y. You, "DiM: Distilling dataset into generative model," 2023, *arXiv:2303.04707*.
[100] L. Zhang et al., "Accelerating dataset distillation via model augmentation," 2022, *arXiv:2212.06152*.
[101] T. Dong, B. Zhao, and L. Liu, "Privacy for free: How does dataset condensation help privacy?," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 5378–5396.
[102] T. Zheng and B. Li, "Differentially private dataset condensation," 2023. [Online]. Available: https://openreview.net/forum?id=H8XpqEkbua_
[103] N. Carlini, V. Feldman, and M. Nasr, "No free lunch in "privacy for free: How does dataset condensation help privacy"," 2022, *arXiv:2209.14987*.
[104] D. Chen, R. Kerkouche, and M. Fritz, "Private set generation with discriminative information," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022. [Online]. Available: https://openreview.net/pdf?id=mxnxRw8jiru
[105] M. Vinaroz and M. J. Park, "Differentially private kernel inducing points (DP-KIP) for privacy-preserving data distillation," 2023, *arXiv:2301.13389*.
[106] N. Tsilivis, J. Su, and J. Kempe, "Can we achieve robustness from data alone?" 2022, *arXiv:2207.11727*.
[107] Y. Wu, X. Li, F. Kerschbaum, H. Huang, and H. Zhang, "Towards robust dataset learning," 2022, *arXiv:2211.10752*.
[108] Y. Liu, Z. Li, M. Backes, Y. Shen, and Y. Zhang, "Backdoor attacks against dataset distillation," 2023, *arXiv:2301.01197*.
[109] D. Zhu et al., "Rethinking data distillation: Do not overlook calibration," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2023, pp. 4935–4945.
[110] S. Ma, F. Zhu, Z. Cheng, and X.-Y. Zhang, "Towards trustworthy dataset distillation," 2023, *arXiv:2307.09165*.



Shiye Lei received the BEng degree in automation engineering from Beihang University, and the MPhil degree in computer science from the University of Sydney, in 2019 and 2022, respectively. He is currently working toward the PhD degree with the School of Computer Science, University of Sydney. His research interests include data-centric AI and knowledge extraction of large-scale datasets.

Dacheng Tao (Fellow, IEEE) is professor of computer science and ARC laureate fellow with the School of Computer Science and the Faculty of Engineering, University of Sydney. His research results in artificial intelligence have expounded in one monograph and more than 200 publications with prestigious journals and prominent conferences, such as *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *International Journal of Computer Vision*, *Journal of Machine Learning Research*, *Artificial Intelligence Journal*, AAAI, IJCAI, NeurIPS, ICML, CVPR, ICCV, ECCV, ICDM, and KDD, with several best paper awards. He received the 2018 IEEE ICDM Research Contributions Award, the 2015 and 2020 Australian Museum Eureka prize, and the 2021 IEEE Computer Society McCluskey Technical Achievement Award. He is a fellow of the AAAS, ACM, and Australian Academy of Science.